

Chapter 14 [Russel, Norvig]

14.1, 14.2 (only sections covered in class), 14.4 (up to 14.4.2)

Chapter 15 [Russel, Norvig]

15.1, 15.2 [Excluding Viterbi algorithm]

Chapter 17 [Russel, Norvig]

17.1, 17.2 [Excluded: 17.2.3], 17.3

Chapter 18 [Russel, Norvig]

up to Section 18.3

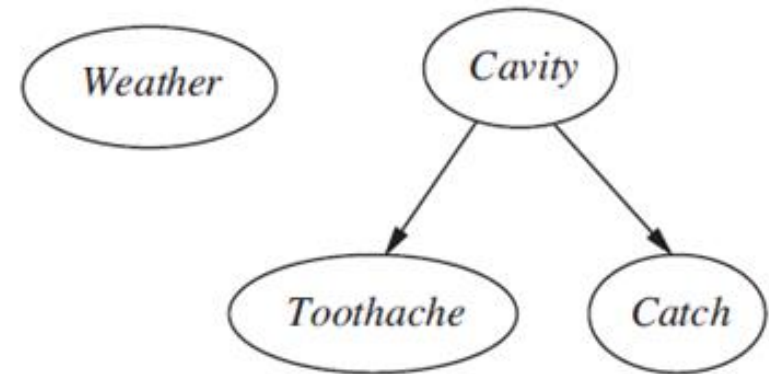
Chapter 14 (AIAMA)

Probabilistic Reasoning

Sukarna Barua
Associate Professor, CSE, BUET

Bayesian Network

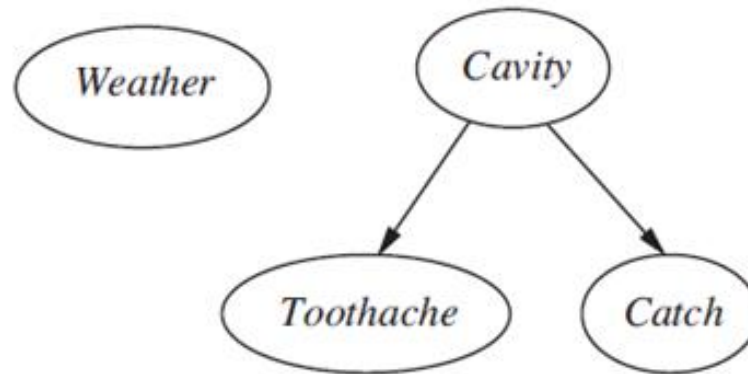
- A Bayesian network (BN) is a probabilistic graphical model.
 - Also known in Bayesian Belief Networks.
- Represented using a *directed acyclic graph (DAG)*.
 - **Nodes:** Represent random variables.
 - **Edges:** Represent conditional relationship (cause and effect) between the random variables.
 - *Cavity causes toothache.*
 - *Cavity causes catch.*
 - *Weather is independent of toothache and cavity.*



Bayesian Network

- **Components of a Bayesian Network:**

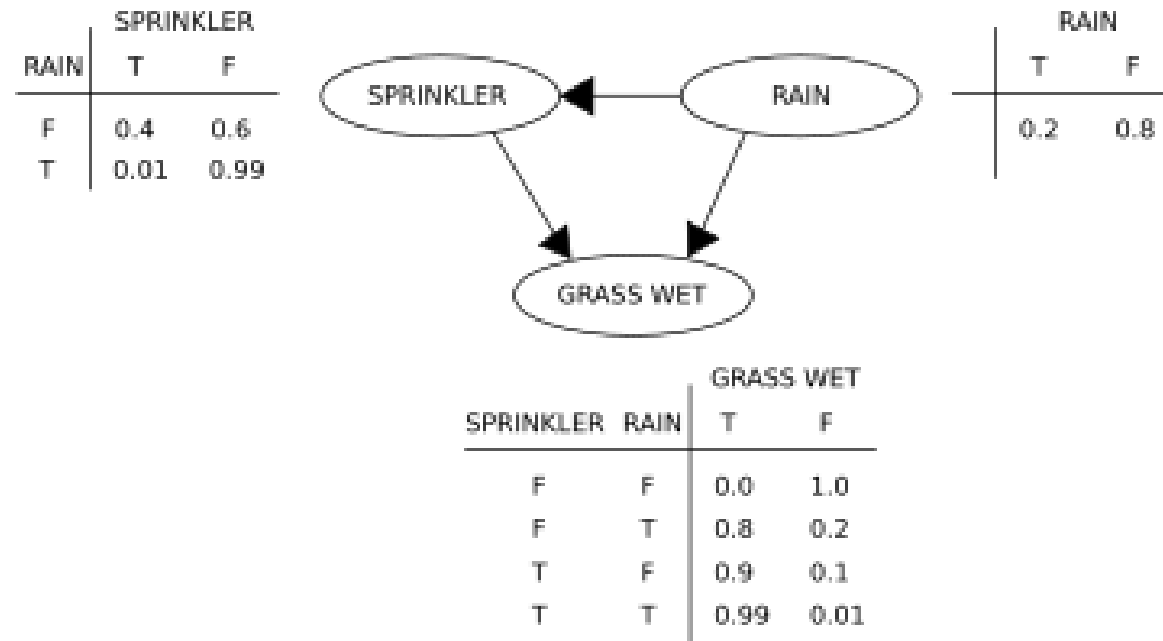
- Nodes: Represent random variables.
- Edges: Represent conditional relationship between the random variables.
 - Edge $A \rightarrow B$: A is a parent of B. B is dependent on A.
 - Two nodes are conditionally independent given their parent.



A simple Bayesian network in which *Weather* is independent of the other three variables and *Toothache* and *Catch* are conditionally independent, given *Cavity*

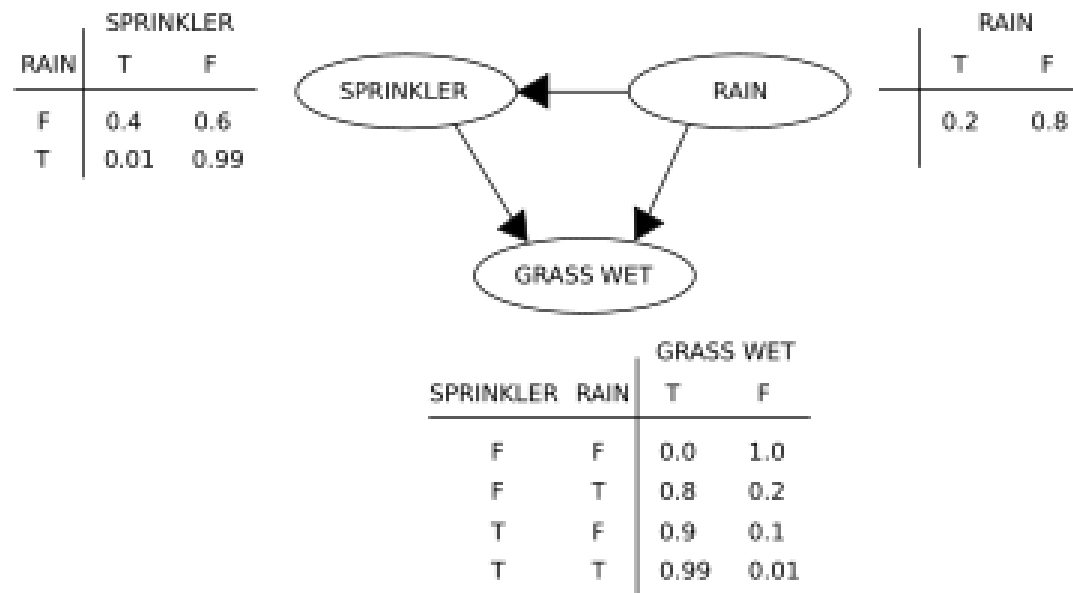
Bayesian Network: CPT

- **CPT**: In a Bayesian network, each node is also annotated with **quantitative probability information** (conditional probability distribution).
- When node has parents: CPT specifies the conditional probability distribution given parent.
- Where node has no parents: CPT specifies the marginal probability distribution.



Bayesian Network: Application

- **Inference:** Determine the probability of unobserved variables based on observed evidences.
 - Causal inference: What is the probability of grass being wet given it is raining?
 - Diagnostic inference: What is the probability of rain given grass is wet?

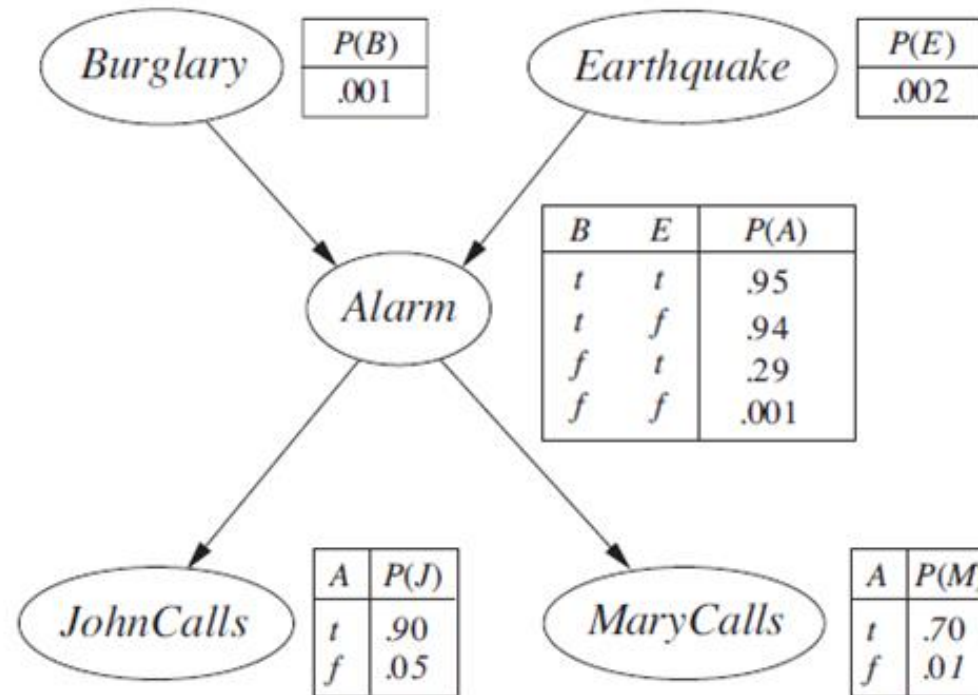


Bayesian Network: Example

- You have a new burglar alarm installed at home. It is fairly reliable at detecting a burglary, but also responds on occasion to minor earthquakes.
 - You also have two neighbors, John and Mary, who have promised to call you at work when they hear the alarm. John nearly always calls when he hears the alarm, but sometimes confuses the telephone ringing with the alarm and calls then, too.
 - Mary, on the other hand, likes rather loud music and often misses the alarm altogether.
- Given the evidence of who has or has not called, we would like to estimate the probability of a burglary.

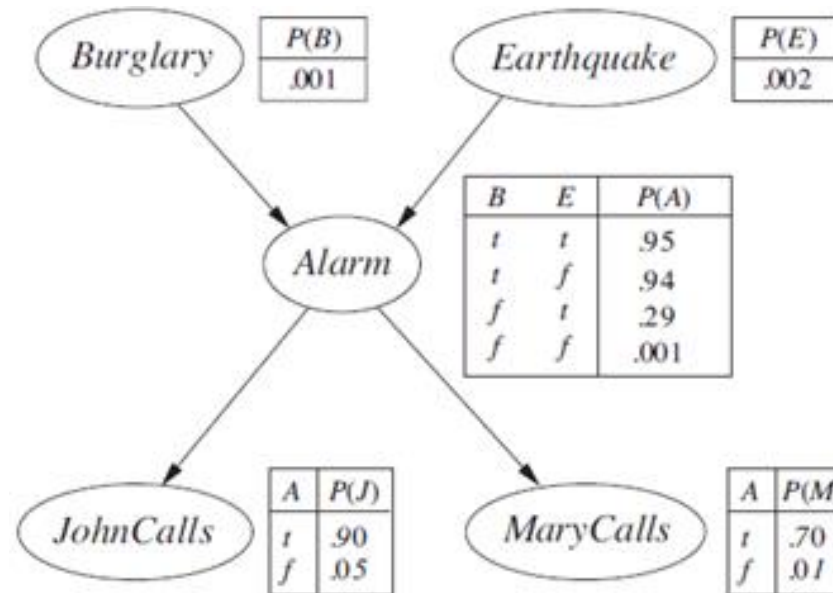
Bayesian Network: Example

- Bayesian network for burglary alarm example:



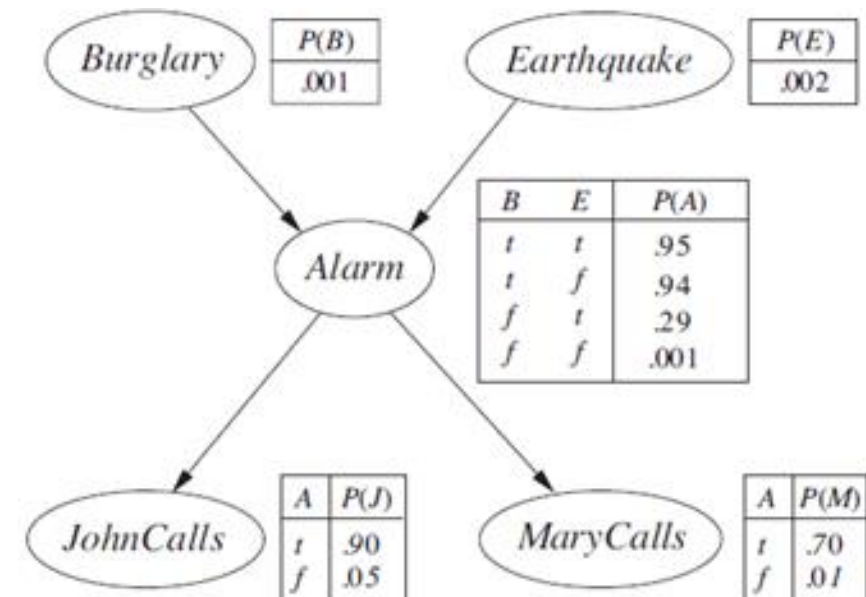
Bayesian Network: Some Observations

- Burglary and Earthquake directly affect the probability of Alarm going off
- John and Marry calls depending only on the Alarm (They know nothing about Burglary or Earthquake)



Bayesian Network: Example

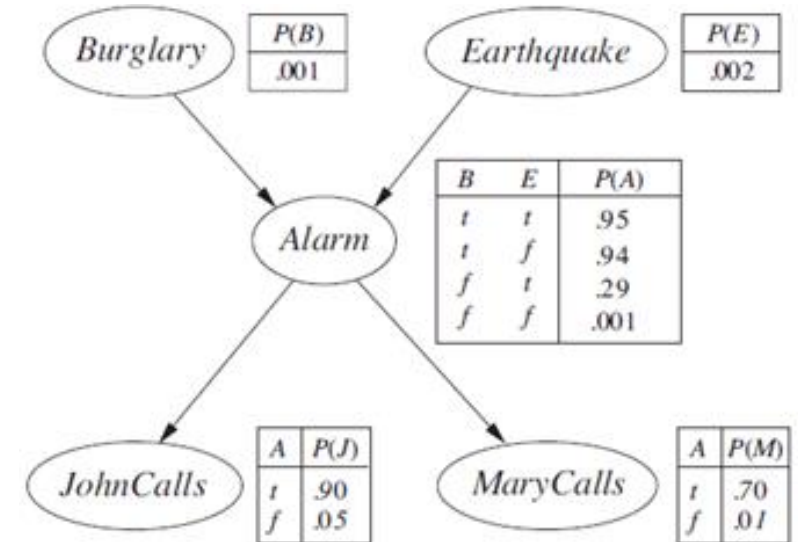
- CPT of a Bayesian Network:
 - Each row specifies the probability of all values (of the node) given a combination of values of the parent nodes
 - Each row must sum to 1.
 - For binary valued nodes, only one (1) probability value is specified for each row. [since the other can be derived]



Bayesian Network: Size of CPTs

- A Boolean node have k Boolean parents. What will be the size of CPT for this node?

- Number of rows:
 - = possible combination of parent values
 - = 2^k
- Number of entries:
 - = $(2^k) \times 1$ [Only one value need to be stored by row]



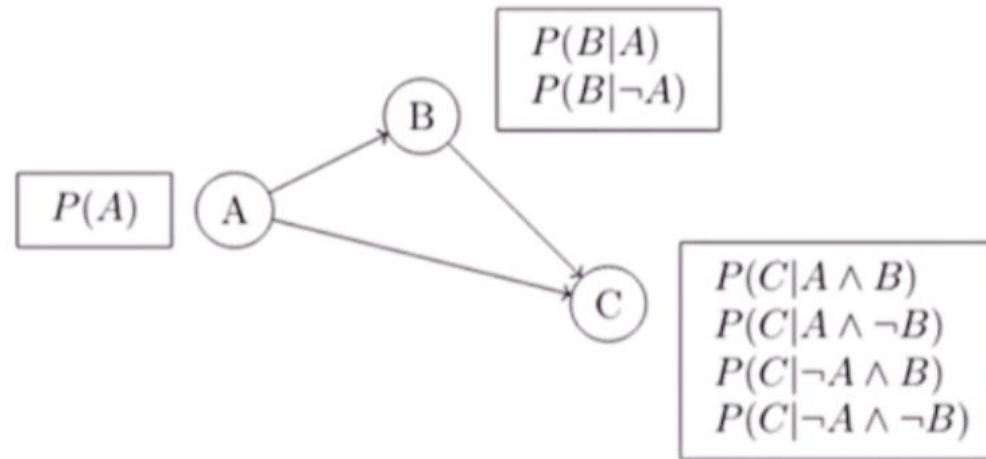
- How many probabilities are needed to represent the entire Bayesian Network?
[Answer: 10]

Bayesian Network: Size of CPTs

- A node A has two parents B and C. A has four states, C is Boolean, and B has three states. What is the size of CPT for node A?
- Answer:
 - Number of rows:
 - = (Number of states of B) $\times$ (Number of states of C)
 - = 3×2
 - = 6
 - Number of entries:
 - = 6×3 [three values need to be stored for each row]
 - = 18

Bayesian Network: Size of CPTs

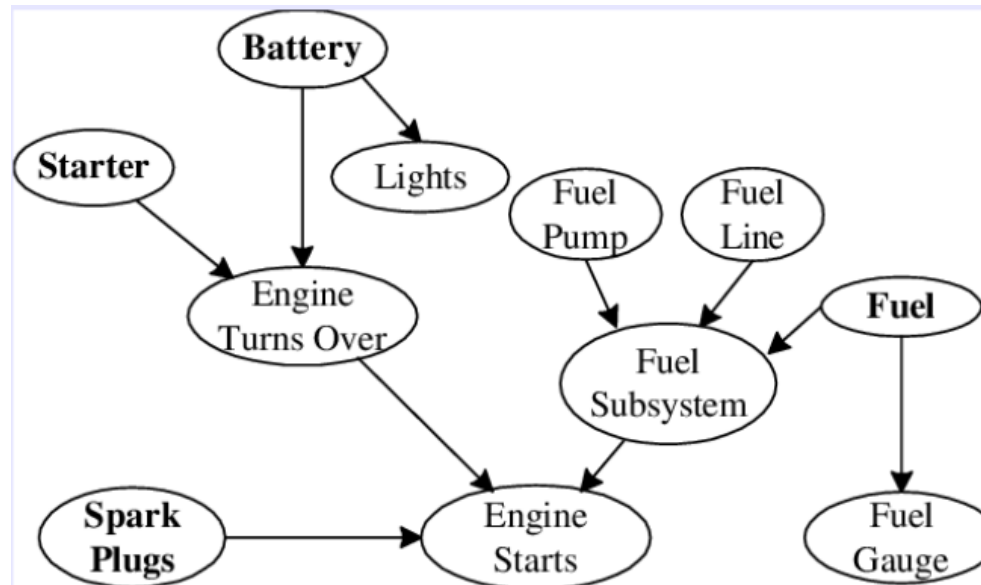
- How many probabilities are needed to represent the Bayesian Network? Assume all are Boolean variables.
 - Answer: $1 + 2 + 4 = 7$



- How many will be needed if B and C are conditionally independent given A ?
 - Answer: $1 + 2 + 2 = 5$.

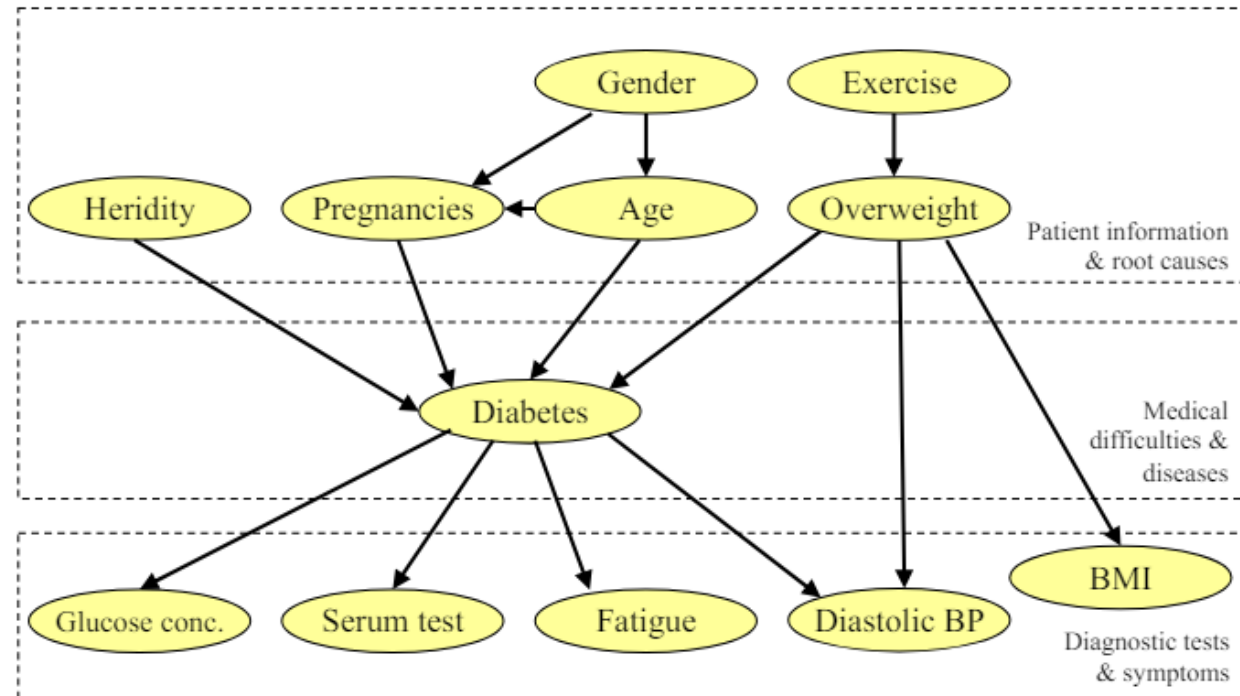
Bayesian Network: More Example

- **A Bayesian Network for car diagnostic system:**
 - we may have problems in a particular part of the car, and that may be caused by many different reasons.
 - The battery might cause the lights to have different problems or it may cause the engine to have problems, and that in turn may affect whether or not the engine can start.



Bayesian Network: More Example

- **A Bayesian Network for diabetes diagnosis.**
 - Symptoms and tests
 - Patient information: the patient's age, weight, medical history, genetics, and so on.

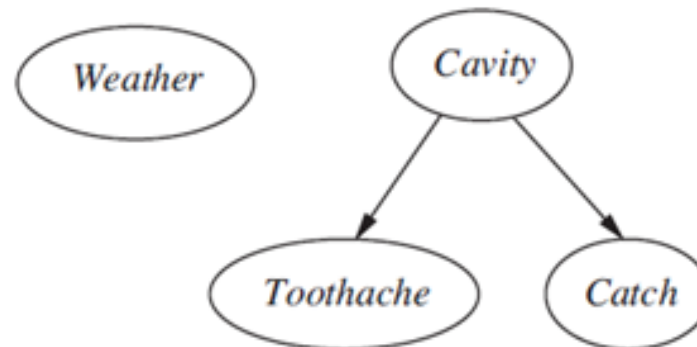


Bayesian Network: Full Specification

- **Nodes:** Each node corresponds to a random variable, which may be discrete or continuous.
- **Edges:** Set of directed links or arrows connects pairs of nodes.
 - If there is an arrow from node X to node Y , X is said to be a parent of Y . The graph has no directed cycles and hence is a directed acyclic graph, or DAG.
- **CPT:** Each node X_i has a conditional probability distribution $P(X_i | Parents(X_i))$ that quantifies the effect of the parents on the node.

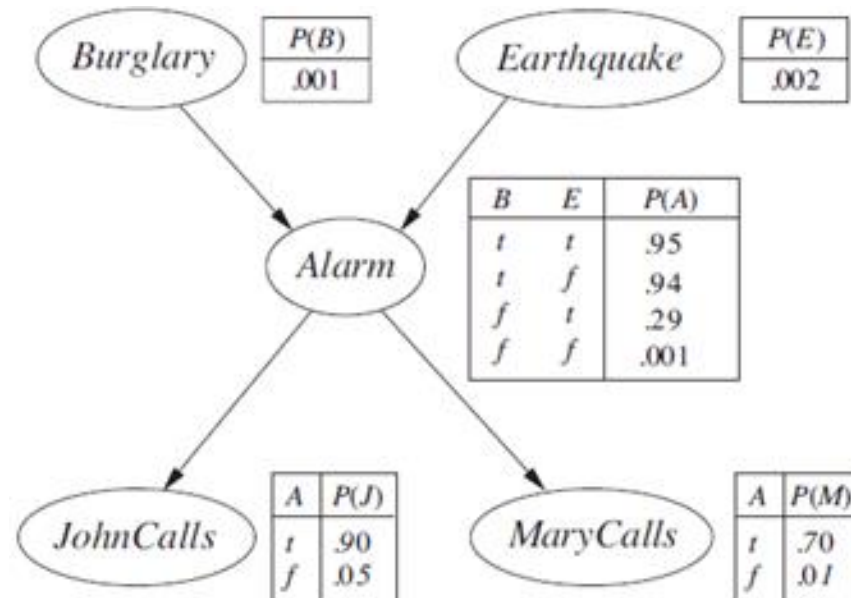
Bayesian Network: Conditional Independence

- In BN, each node is conditionally independent of others given its parents!
 - *To know the probability of a node, you only need to know its parents!*
 - *No edge from $X \rightarrow Y$: X is conditionally independent of Y given X 's parents!*
- BN implicitly defines conditional independence of random variables
 - *Weather* is Independent of other variables.
 - *Toothache* and *Catch* are conditionally independent given *Cavity*.



Bayesian Network: Conditional Independence

- Implicitly defines conditional independence of random variables
 - JohnCalls is Independent of MarryCalls given Alarm.
 - MarryCalls is independent of EarthQuake given Alarm.
 - Is Burglary independent of Earthquake? [Yes, Why?].



Bayesian Network: Conditional Independence

- Implicitly defines conditional independence of random variables.
 - Are Burglary and Watson independent? [No]
 - Are Burglary and Watson independent given Alarm? [Yes]

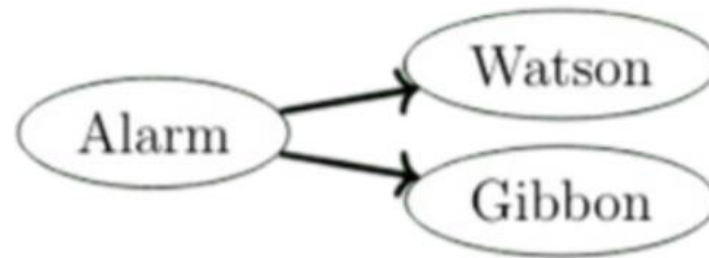


Bayesian Network: Conditional Independence

- Implicitly defines conditional independence of random variables!

- Are Watson and Gibbon independent? [No]

[If Watson is calling, it is more likely that Alarm is off, which means that Gibbon is more likely to call]

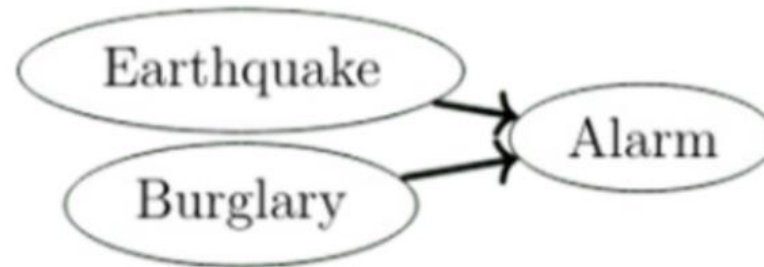


Bayesian Network: Conditional Independence

- Implicitly defines conditional independence of random variables!

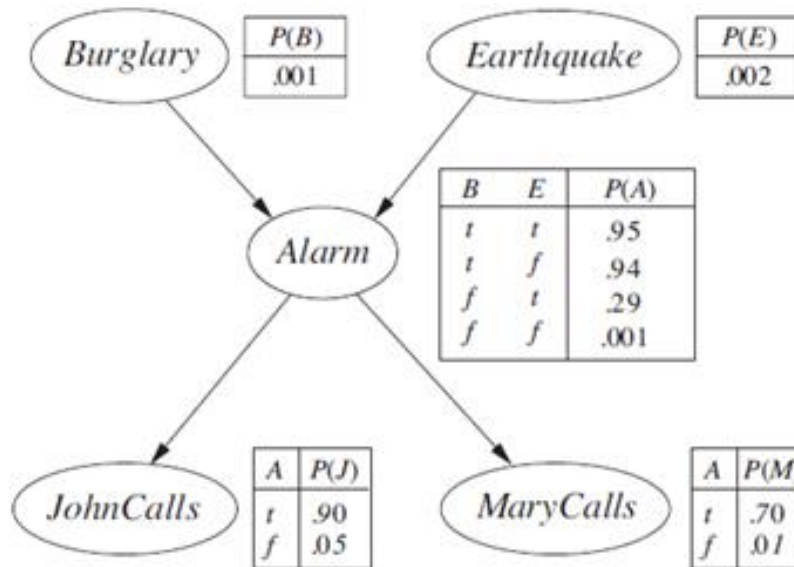
- Are Earthquake and Burglary independent? [Yes]
- Are Earthquake and Burglary independent given Alarm? [No]

[If Earthquake is happening, then it is less likely that the Alarm is caused by Burglary.]



Bayesian Network: Full Joint Distribution

- Why you need joint distribution?
 - *You can infer any probability from the joint!*
- *Bayesian network + conditional probability distribution* for each variable, given its parents specify full joint distribution for all the variables.



Bayesian Network: Full Joint Distribution

- Given a Bayesian Network with variables/nodes $x_1, x_2, \dots, x_n$
- Find the joint probability $P(x_1, x_2, \dots, x_n)$
- We can show that:

$$\begin{aligned} P(x_1, \dots, x_n) &= P(x_n | x_{n-1}, \dots, x_1) P(x_{n-1}, \dots, x_1) . \\ &= P(x_n | x_{n-1}, \dots, x_1) P(x_{n-1} | x_{n-2}, \dots, x_1) \cdots P(x_2 | x_1) P(x_1) \\ &= \prod_{i=1}^n P(x_i | x_{i-1}, \dots, x_1) . \end{aligned}$$

Bayesian Network: Full Joint Distribution

- If nodes are ordered such that all parent nodes of $x_i \in \{x_{i-1} \dots, x_1\}$, then the full joint distribution can be written as:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{Parent}(X_i))$$

- The joint probability is the product of the individual (conditional) probabilities of each node!
- Bayesian network is a compact way of representation a joint distribution!
 - We just need to find a topological sorting of the nodes for the above equation to become correct!

Bayesian Network: Compute Full Joint Distribution

- Calculate the probability that the alarm has sounded, but neither a burglary nor an earthquake has occurred, and both John and Mary call.

- We have following values:

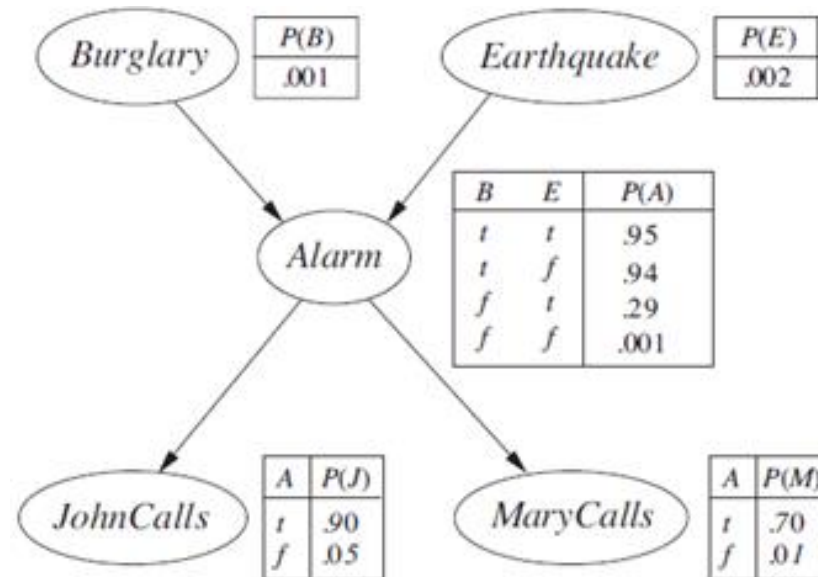
a = alarm sounds

$\neg b$ = no burglary

$\neg e$ = no earthquake

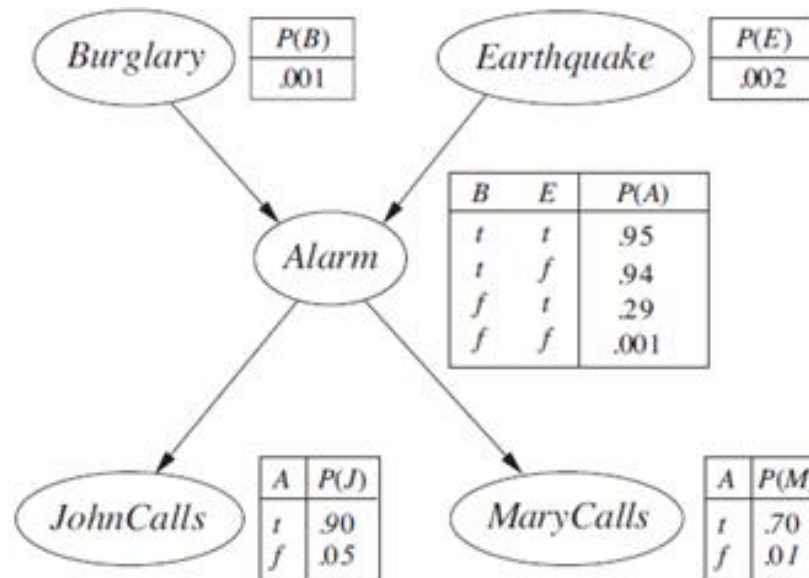
j = John calls

m = Marry calls



Bayesian Network: Compute Full Joint Distribution

- Calculate the probability that the alarm has sounded, but neither a burglary nor an earthquake has occurred, and both John and Mary call.
 - We need to calculate $P(a, \neg b, \neg e, j, m)$.
 - We can compute the full joint distribution using CPTs.



Bayesian Network: Compute Full Joint Distribution

$$P(a, \neg b, \neg e, j, m)$$

$$= P(j, m, a, \neg b, \neg e) \quad [\text{rearrange as per topological sorting}]$$

$$= P(j|m, a, \neg b, \neg e)P(m, a, \neg b, \neg e) \quad [P(a, b) = P(a|b)P(b)]$$

$$= P(j|a)P(m, a, \neg b, \neg e) \quad [j \text{ is independent of other variables given } a]$$

$$= P(j|a)P(m|a)P(a, \neg b, \neg e)$$

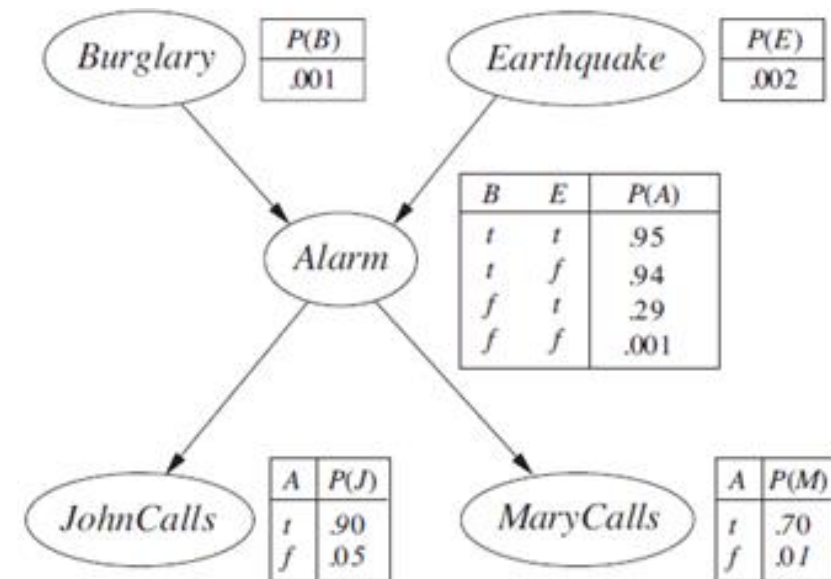
$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b, \neg e)$$

$$= P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

[b and e are independent of each other]

$$= 0.9 * 0.7 * 0.001 * 0.999 * 0.998$$

$$= 0.000628$$



Chapter 14 (AIAMA)

Probabilistic Reasoning

Sukarna Barua

Associate Professor, CSE, BUET

Bayesian Network: Exact Inference

- **Consider the query:** $P(\text{Burglary} | \text{JohnCalls} = \text{ture}, \text{MaryCalls} = \text{true})$
- We need to compute: $P(B|j, m)$ [compute for both b and $\neg b$]
- Express in terms of joint distributions [we already know how to compute joint distribution]

$$P(B|j, m) = \frac{P(B, j, m)}{P(j, m)} = \alpha P(B, j, m)$$

where α is the normalizing constant [we can find the value of α later]

Bayesian Network: Exact Inference

- How to compute $P(B, j, m)$?
- Add all hidden variables to get full joint distribution:

$$P(B, j, m) = \sum_A \sum_E P(B, j, m, A, E) = \sum_A \sum_E P(j|A)P(m|A)P(A|B, E)P(B)P(E)$$

- Query variable: B
- Evidence variables: J, M
- Hidden variables: A, E [*Need to sum over hidden variables to get the marginal!*]

Bayesian Network: Exact Inference

- Finally, we get the following:

$$P(B|j, m) = \alpha \sum_A \sum_E P(j|A)P(m|A)P(A|B, E)P(B)P(E)$$

- Compute $P(b|j, m)$ and $P(\neg b|j, m)$ [expression has α]
 - Find α [Using the eq. $P(b|j, m) + P(\neg b|j, m) = 1$]
- **Question:** How many operations (multiplications + additions)?
 - Answer: [For each $P(B|j, m)$] Mult: 16, Sum: 4, Total = 20 operations]

Bayesian Network: Exact Inference

- Computing using the join distribution naively:

$$P(B|j, m) = \alpha \sum_A \sum_E P(j|A)P(m|A)P(A|B, E)P(B)P(E)$$

- Required operations: 19 [3 additions and 16 multiplications]
 - 4 terms and each term needs 4 multiplications = total 16 multiplications
 - Sum the 4 terms: need 3 sums
- **Can we improve?**
 - Yes, move sums inside [*closest to the factors having the hidden variable*]
 - Sums are evaluated early and gets multiplied once [*rather than every time during joint calculation*]

Bayesian Network: Exact Inference

- Move summations inside the express to factorize:

$$P(B|j, m) = \alpha \sum_a \sum_e P(j|a)P(m|a)P(a|B, e)P(B)P(e)$$

- Two alternate ways to do it:
 - Option 1: $P(B|j, m) = \alpha P(B) \sum_A P(j|A)P(m|A) \sum_E P(A|B, E)P(E)$
 - Option 2: $P(B|j, m) = \alpha P(B) \sum_E P(E) \sum_A P(A|B, E)P(j|A)P(m|A)$
 - Which one is more efficient in terms of operations?

Bayesian Network: Exact Inference

- Expand the sums and compute the number of multiplications and additions:

- $$P(B|j, m) = \alpha P(B) \sum_A P(j|A)P(m|A) \sum_E P(A|B, E)P(E)$$

$$= \alpha P(B) (P(j|a)P(m|a) \sum_E P(a|B, E)P(E) + P(j|\neg a)P(m|\neg a) \sum_E P(\neg a|B, E)P(E))$$

[Expanding $\sum_A$]

$$= \alpha P(B) (P(j|a)P(m|a) (P(a|B, e)P(e) + P(a|B, \neg e)P(\neg e)) +$$

$$P(j|\neg a)P(m|\neg a) (P(\neg a|B, e)P(e) + P(\neg a|B, \neg e)P(\neg e)))$$

[Expanding $\sum_E$]

Bayesian Network: Exact Inference

- **Which one is more efficient?** Assume $B = \text{true}$ and compute both.
- Equation 1: $P(B|j, m) = \alpha P(B) \sum_A P(j|A)P(m|A) \sum_E P(A|B, E)P(E)$
 - Required # of multiplications: 9, # of sums = 3, total = 12
- Equation 2: $P(B|j, m) = \alpha P(B) \sum_E P(E) \sum_A P(A|B, E)P(j|A)P(m|A)$
 - Required # of multiplications: 11, # of sums = 3, total = 14
- So order matters! However, finding optimal order is difficult!

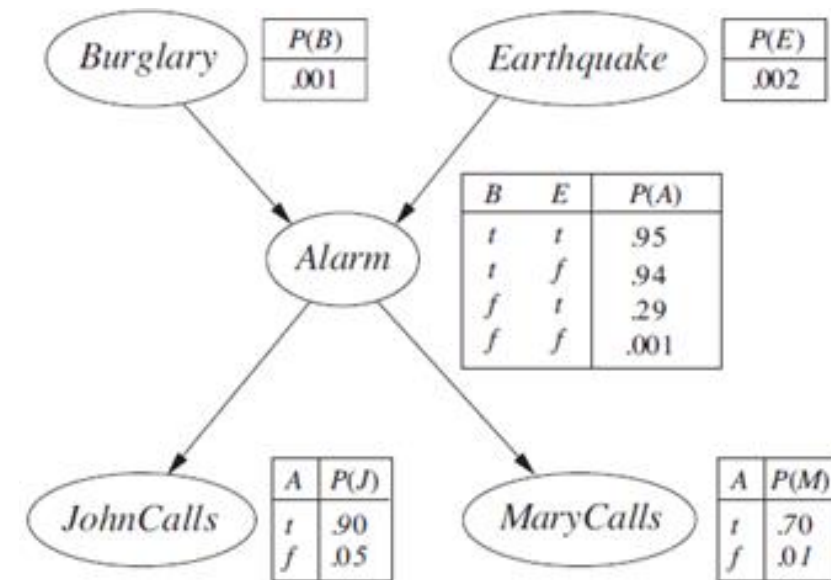
Bayesian Network: Exact Inference

- Calculate: $P(B|j, m) = \alpha P(B) \sum_A P(j|A) P(m|A) \sum_E P(A|B, E) P(E)$

- Using the CPT data, we calculate:

$$P(b|j, m) = \alpha \times 0.00059224$$

$$P(\neg b|j, m) = \alpha \times 0.0014919$$



- Hence,

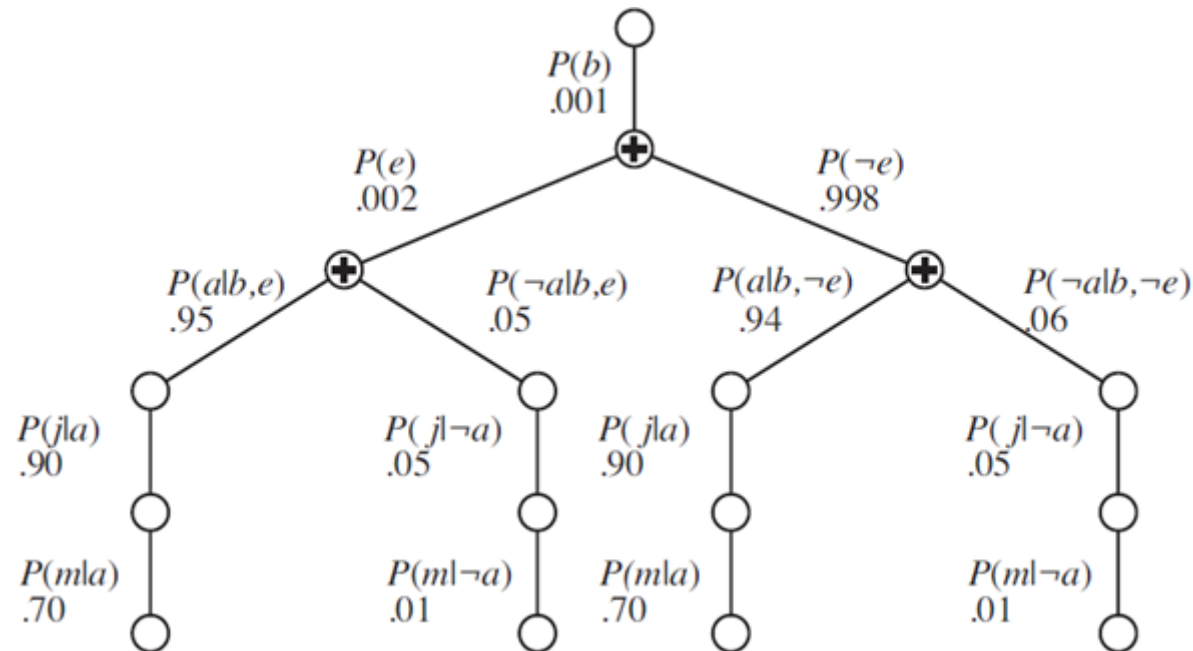
$$\begin{aligned}
 P(B|j, m) &= \alpha \times (0.00059224, 0.0014919) \\
 &= (0.284, 0.716) \text{ [After normalizing and calculating } \alpha \text{]}
 \end{aligned}$$

Bayesian Network: Exact Inference

- An evaluation tree is shown for the expression:

$$P(B|j, m) = \alpha P(B) \sum_E P(E) \sum_A P(A|B, E) P(j|A) P(m|A)$$

- Blank circled nodes represent multiplication [Notice the repetition of sub-paths]



Bayesian Network: Exact Inference

- **Problem:** Repeated subexpression evaluation
- The following products are computed twice in two paths of the tree!
 - $P(j \mid a)P(m \mid a)$
 - $P(j \mid \neg a)P(m \mid \neg a)$
- **Problem:** Repeated subexpression evaluation
- The following products are computed twice in two paths of the tree!
 - $P(j \mid a)P(m \mid a)$
 - $P(j \mid \neg a)P(m \mid \neg a)$

Bayesian Network: Variable Elimination Algorithm

- Consider the following expression: $P(B|j, m)$.
 - Query variable: B
 - Evidence variable: J, M
 - Hidden variable: A, E
- Calculating the posterior distribution of one or more query variables given some evidence is in #NP.
- We will use **variable elimination** algorithm.
 - Solves repeated subexpression problem using *dynamic programming*.

Bayesian Network: Variable Elimination Algorithm

- Variable Elimination (VE) Algorithm
 - Step 1: Compute factors for each node of Bayesian Network and instantiate/restrict evidence variables.
 - Step 2: For some order of variables $(V_1, V_2, \dots, V_N)$:
 - (Step 2a) If variable V is a hidden variable, then multiply all the factors of V .
 - (Step 2b) Sum out V .
 - Step 3: Multiply remaining factors.
 - Step 4: Normalize the final factor to eliminate α .

Bayesian Network: Variable Elimination Algorithm

- Consider the following expression: $P(B|j, m)$.
 - Query variable: B
 - Evidence variable: J, M
 - Hidden variable: A, E
- We will use variable elimination algorithm to calculate
- Calculating the posterior distribution of one or more query variables given some evidence is in #NP.

Bayesian Network: Variable Elimination Algorithm

- Query: $P(B|j, m)$.
- Joint probability expression of BN:

$$P(B, E, A, J, M) = P(B)P(E)P(A|B, E)P(J|A)P(M|A)$$

- **Step 1:** Compute all the factors of BN.
- The BN has five factors [*note the joint probability expression*]:
 - $f_1(B) = P(B)$
 - $f_2(E) = P(E)$
 - $f_3(A, B, E) = P(A|B, E)$
 - $f_4(J, A) = P(J|A)$
 - $f_5(M, A) = P(M|A)$

Variable Elimination Algorithm: Compute Factors

- Query: $P(B|j, m)$.
- **Step 1a:** Compute all the factors of BN. [*don't restrict evidences now*]

$$f_1(B) = P(B)$$

T	0.001
F	0.999

$$f_2(E) = P(E)$$

T	0.002
F	0.998

$$f_3(A, B, E) = P(A|B, E)$$

T	T	T	0.95
T	T	F	0.94
T	F	T	0.29
T	F	F	0.001
F	T	T	0.05
F	T	F	0.06
F	F	T	0.71
F	F	F	0.999

$$f_4(J, A) = P(J|A)$$

T	T	0.90
T	F	0.05
F	T	0.10
F	F	0.95

$$f_5(M, A) = P(M|A)$$

T	T	0.70
T	F	0.01
F	T	0.30
F	F	0.99

Variable Elimination Algorithm: Restrict Factors

- Query: $P(B|j, m)$.
- **Step 1b:** Instantiate/restrict evidence variables in all factors [j and m].
 - Restrict J to true in $f_4(J, A)$. [*Keep only rows where $J = \text{true}$*]
 - Note that J is removed from factor expression.

$f_4(J, A) = P(J A)$				$f_4(A)$	
T	T	0.90		T	0.90
T	F	0.05		F	0.05
F	T	0.10			
F	F	0.95			

Variable Elimination Algorithm: Restrict Factors

- Query: $P(B|j, m)$.
- **Step 1b:** Instantiate/restrict evidence variables in all factors. [j and m].
 - Restrict M to true in $f_5(M, A)$. [Keep only rows where $M = \text{true}$]

$f_5(M, A) = P(M A)$					$f_5(A)$
T	T	0.70		T	0.70
T	F	0.01		F	0.01
F	T	0.30			
F	F	0.99			

Variable Elimination Algorithm: Multiply Factors

- Query: $P(B|j, m)$.
- **Step 2:** Select every variable V (in any order) which is hidden [Here, A and E]
 - Multiply all factors of V and then sum-out the variable.
 - Suppose we select in this order: E, A .
 - First we select E
 - Multiple all factors having E and get a new factor: $f_6(A, B, E) = f_2(E) \times f_3(A, B, E)$
 - Sum-out E and get a new factor: $f_7(A, B) = \sum_E f_6(A, B, E)$
 - Note that after summing-out, E is removed from factor-argument.

Bayesian Network: Multiply Factors

- **Step 2a:** Multiple all factors having E and get a new factor:

$$f_6(A, B, E) = f_2(E) \times f_3(A, B, E)$$

- Factor multiplication is a point-wise multiplication of elements in two factors:
 - Multiply rows which have matching values of the variables.
 - Number of rows in the new factor = 2^N where N is number of total variables in the new factor:
 - Multiplying $f_1(A, B)$ and $f_2(B, C)$ will give us $f_3(A, B, C)$ which will have 8 rows.

Bayesian Network: Multiply Factors

- For variable E :
 - **Step 2a:** Multiple all factors having E and get a new factor:
 - **Multiply:** $f_6(A, B, E) = f_2(E) \times f_3(A, B, E)$

$$f_2(E) = P(E)$$

T	0.002
F	0.998

×

$$f_3(A, B, E) = P(A|B, E)$$

T	T	T	0.95
T	T	F	0.94
T	F	T	0.29
T	F	F	0.001
F	T	T	0.05
F	T	F	0.06
F	F	T	0.71
F	F	F	0.999



$$f_6(A, B, E)$$

T	T	T	0.0019
T	T	F	0.93812
T	F	T	0.00058
T	F	F	0.000998
F	T	T	0.0001
F	T	F	0.05988
F	F	T	0.00142
F	F	F	0.997002

Bayesian Network: Multiply Factors

- For variable E:
 - **Step 2b:** Sum-out resulting factor for E and get a new factor:
 - **Sum-out E :** $f_7(A, B) = \sum_E f_6(A, B, E)$ [sum corresponding rows that have T and F for E]

$$f_6(A, B, E)$$

T	T	T	0.0019
T	T	F	0.93812
T	F	T	0.00058
T	F	F	0.000998
F	T	T	0.0001
F	T	F	0.05988
F	F	T	0.00142
F	F	F	0.997002

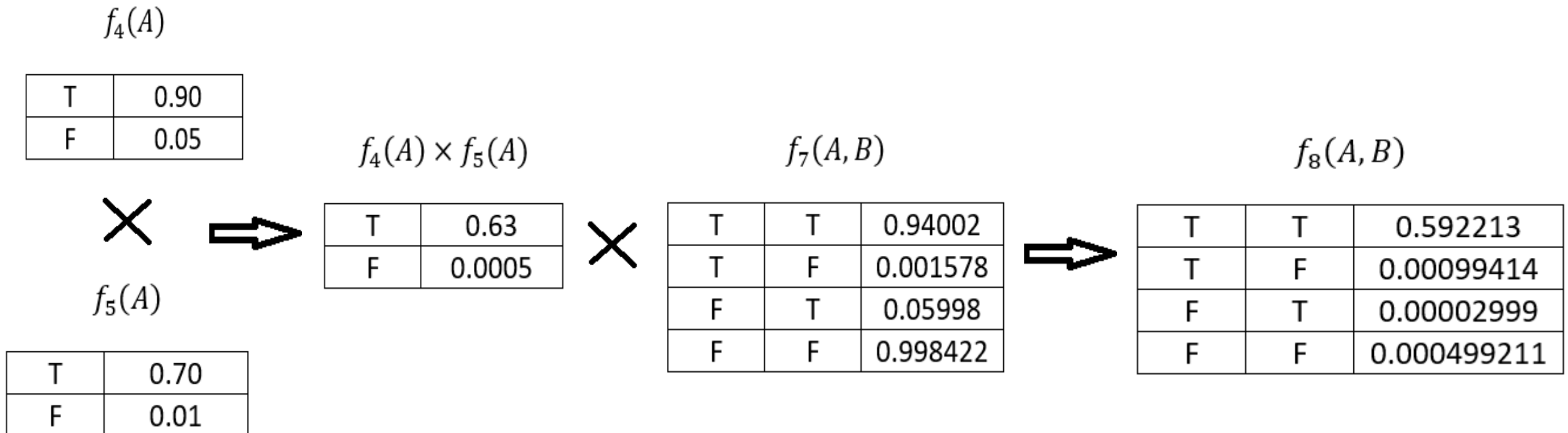


$$f_7(A, B)$$

T	T	0.94002
T	F	0.001578
F	T	0.05998
F	F	0.998422

Bayesian Network: Multiply Factors

- For variable A:
 - Step 2a:** Multiple all factors having A and get a new factor:
 - Multiply:** $f_8(A, B) = f_4(A) \times f_5(A) \times f_7(A, B)$



Bayesian Network: Multiply Factors

- For variable A:
 - **Step 2b:** Sum-out resulting factor for E and get a new factor:
 - **Sum-out A:** $f_9(B) = \sum_A f_8(A, B)$ [sum corresponding rows that have T and F for E]

$f_8(A, B)$					$f_9(B)$
T	T	0.592213	⇒	T	0.59224299
T	F	0.00099414		F	0.001493351
F	T	0.00002999			
F	F	0.000499211			

Bayesian Network: Multiply Remaining Factors

- Query: $P(B|j, m)$.
- **Step 3:** Multiply remaining factors: $f_{10}(B) = f_1(B) \times f_9(B)$

$$f_1(B) = P(B) \quad \times \quad f_9(B) \quad \Rightarrow \quad f_{10}(B)$$

T	0.001
F	0.999

T	0.59224299
F	0.001493351

T	0.000592
F	0.001492

Bayesian Network: Normalize Result

- Query: $P(B|j, m)$.
- **Step 4:** Normalize the final factor to eliminate α .

$f_{10}(B)$			$f_{11}(B)$	
T	0.000592	$\Rightarrow$	T	0.284172
F	0.001492		F	0.715828

- **Answer:** $P(b|m, j) = 0.284$, $P(\neg b|m, j) = 0.716$

Bayesian Network: Variable Elimination Algorithm

- Algorithm pseudocode [RN book]
 - Note: Step 1 (initialization) is missing in the pseudocode [it is implied in the MAKE-FACTOR function which is not a good approach!]

```
function ELIMINATION-ASK( $X, \mathbf{e}, bn$ ) returns a distribution over  $X$   
  inputs:  $X$ , the query variable  
            $\mathbf{e}$ , observed values for variables  $\mathbf{E}$   
            $bn$ , a Bayesian network specifying joint distribution  $\mathbf{P}(X_1, \dots, X_n)$   
  
   $factors \leftarrow []$   
  for each  $var$  in ORDER( $bn.VARS$ ) do  
     $factors \leftarrow [MAKE-FACTOR(var, \mathbf{e}) | factors]$   
    if  $var$  is a hidden variable then  $factors \leftarrow SUM-OUT(var, factors)$   
  return NORMALIZE(POINTWISE-PRODUCT( $factors$ ))
```

Bayesian Network: Irrelevant Variable

- Consider the query: $P(\text{JohnCalls} | \text{Burglary} = \text{true})$
- Incorporating all hidden variables of the Bayesian Network, we get:

$$P(J|b) = \alpha P(J, b) = \alpha \sum_e \sum_a \sum_m P(j, m, a, b, e) = \sum_e \sum_a \sum_m P(J|a)P(m|a)P(a|b, e)P(b)P(e)$$

Bayesian Network: Irrelevant Variable

- A simple rearrangement of the expression would yield us:

$$\mathbf{P}(J | b) = \alpha P(b) \sum_e P(e) \sum_a P(a | b, e) \mathbf{P}(J | a) \sum_m P(m | a) .$$

- Note that the inner term $\sum_m P(m|a)$ will evaluate to 1. So it is eliminated from the expression entirely [Irrelevant variable!]
- *Variable elimination algorithm will automatically sum-out m from the expression!*
 - Why? [At some point, m will be considered as a hidden variable and the summation over m will eliminate it]

Bayesian Network: Irrelevant Variable

- In general, we can remove any leaf node that is not a query variable or an evidence variable.
- After removal of initial leaves, there may be some more leaf nodes, and these too may be irrelevant and removed [continue this process].
- *Every variable that is not an ancestor of a query variable or evidence variable is irrelevant to the query.*
- *Finding the optimal ordering of hidden variables for VE is NP-Hard!*

Chapter 15 (AIAMA)

Probabilistic Reasoning Over Time

Sukarna Barua

Associate Professor, CSE, BUET

Temporal Probabilistic Models

- Static world (as we considered in Bayesian network):
 - Random variables have a fixed number of states/values.
 - Values of Random variables doesn't change over time
- Dynamic world (time is an important factor):
 - Random variables have a fixed number of states/values.
 - Values of Random variables change over time.

Temporal Probabilistic Models

- Dynamic world has a state at time t
 - State is composed of a set of random variables X_t
 - A snapshot of the state at time t is a set of values of X_t
- State is not observable
 - State is not directly observable.
 - A set of evidence variables E_t are observable at time t [evidences depends on state]
 - We may infer which state we are in from the evidence!

Temporal Probabilistic Models: Example

You want to know whether you have infection at time step t . You can measure fever, headache, stomachache at time step t .

- $X_t: \{Infection_t\}$
 - Values: Yes/No [*Unobservable by agent, it is hidden*]
- $E_t: \{Fever_t, Stomachache_t, Headache_t\}$
 - Values: Yes/No [*Observable by agent*]

Temporal Probabilistic Models

In a temporal probabilistic model, agent have:

- Environment: Partially observable
- Belief state: What is the current state as agent maintains/believes?
- Transition model: How the environment might evolve in the next time step
- Sensor model: How the observable events happen at world state?
- Decision: How the agent take action?
 - Evidence $\rightarrow$ Belief state $\rightarrow$ Decision

Hidden Markov Models

- A temporal probabilistic model may be called a Hidden Markov Model (HMM) when the state is represented by a discrete random variable:
- X_t : A single state variables at time t
 - Unobservable by agent [*hidden from the agent*]
- E_t : Set of evidence variables
 - Observable by agent [*known through percepts*]

Hidden Markov Models

- What happens if world state has multiple random variables?
 - Multiple random variables may be mapped to a single random variable
 - Example: $\langle \text{Burglary}, \text{Earthquake} \rangle$ makes up agent state both are Boolean.
 - Construct a single variable $\langle \text{BE} \rangle$ with four values $\{0,1,2,3\}$ where
 - 0 means Burglary=T and Earthquake = T
 - 1 means Burglary=T and Earthquake = F
 - 2 means Burglary=F and Earthquake = T
 - 3 means Burglary=F and Earthquake = F

Hidden Markov Models: Example

A security guard inside a building needs to know whether it's raining outside. He can only see if someone coming in with/without an umbrella.

- $X_t: \{Rain_t\}$
 - Values: Yes/No [Unobservable by agent]
- $E_t: \{Umbrella_t\}$
 - Values: Yes/No [Observable by agent]

Transition Model

- Specifies the probability distribution of the state at time t , given the previous states:

$$P(X_t | X_{1:t-1})$$

- Assume the size of CPT when t is large [*exponentially large*]
- Problematic as number of time steps increases
- Not practical as current state may depend only on few previous states

Markov Assumption for Transition Model

- Assumption: Current state is independent of all states $X_{1:t-k-1}$ given the previous k number of states $X_{t-k:t-1}$:

$$P(X_t | X_{1:t-1}) = P(X_t | X_{t-k:t-1})$$

- Markov Process: Process satisfying Markov assumption.
 - Also known as Markov chains.
 - After Russian mathematician Andrei Markov

Order of Markov Process

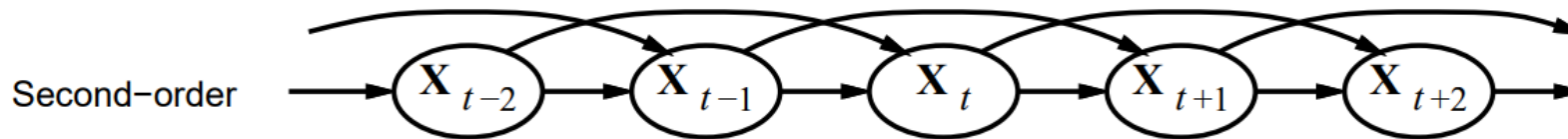
- First Order Markov Process:

- Current state is independent of all other states given only the previous state
- $P(X_t|X_{1:t-1}) = P(X_t|X_{t-1})$
- Transition model is a conditional distribution $P(X_t|X_{t-1})$



- For a second order Markov Process:

- Transition model is a conditional distribution $P(X_t|X_{t-1}, X_{t-2})$



First Order Markov Process

- Stationary process: transition model do not change over time steps
 - $P(X_t|X_{t-1})$ is same for all time steps t .
 - $P(X_2|X_1) = P(X_3|X_2) = \dots$
 - $P(X_t = x_j | X_{t-1} = x_i) = a_i[j]$
- [a_{ij} is the probability of state transitioning from x_i to x_j]*

Sensor/Emission Model

- Evidence values depend on current state as well as all previous states and evidence values
- Probability distribution of events E_t :

$$P(E_t | X_{1:t}, E_{1:t-1})$$

- What is the probability that $Umbrella_t = true$ given all previous state and evidence values?
- What is the size of CPT when t is large? [*exponentially large*]
- Not practical from computational perspective

Markov Assumption for Sensor Model

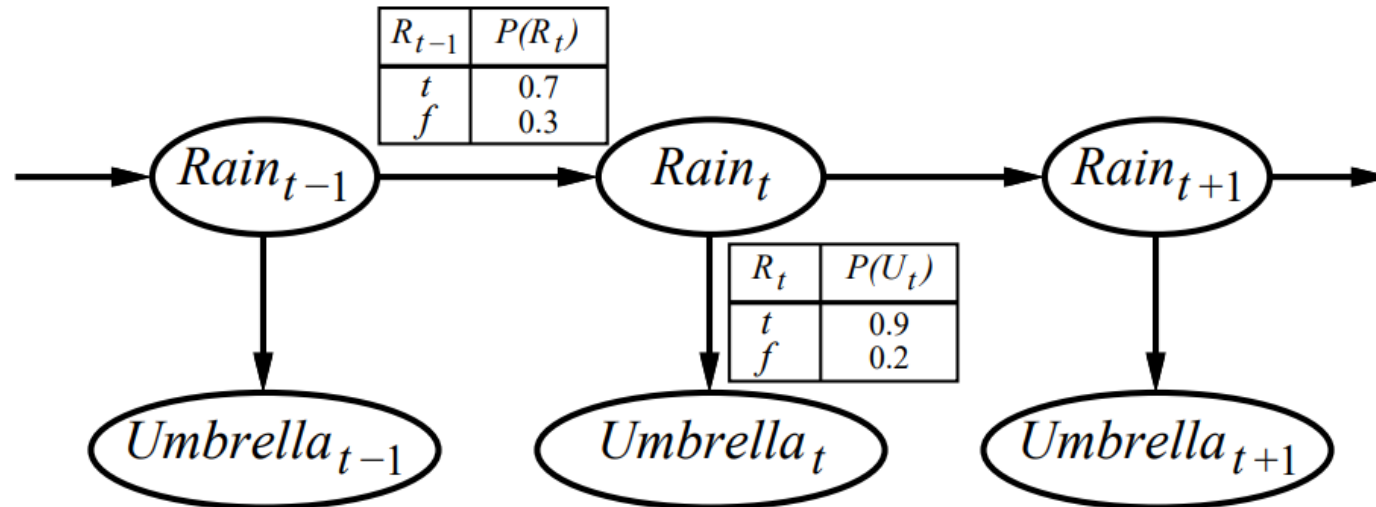
- Assumption: Evidence at time t is independent of all previous states and events given the state at time t (current state).

$$P(E_t | X_{1:t}, E_{1:t-1}) = P(E_t | X_t) \quad [\text{evidence depend only on current state}]$$

- Evidence only depend on current state and is independent of all previous states and evidences
- $P(E_t = e_k | X_t = x_i) = o_{ik}$ [probability of emitting output o_k from state x_i]
- Also known as Observation/Emission Model

Example Markov Process

- For the umbrella example:
 - Transition model: $P(R_t|R_{t-1})$, sensor model: $P(U_t|R_t)$



Complete/Full Joint Distribution

- We have
 - $P(X_t|X_{t-1})$ [*transition model*]
 - $P(E_t|X_t)$ [*sensor model*]
- We also need
 - $P(X_1)$: The prior probability distribution of states at time step $t = 1$
- Complete joint distribution can be computed as:

$$P(X_{1:t}, E_{1:t}) = P(E_{1:t}|X_{1:t})P(X_{1:t}) = \prod_{i=1}^t P(E_i|X_i)P(X_i|X_{i-1})$$

[Assume $P(X_1|X_0) = P(X_1)$ for notational convenience]

Complete/Full Joint Distribution

- Complete joint distribution derivation:

$$\begin{aligned}P(X_{1:t}, E_{1:t}) &= P(E_{1:t}|X_{1:t})P(X_{1:t}) \\&= P(E_t|E_{1:t-1}, X_{1:t})P(E_{1:t-1}|X_{1:t})P(X_t|X_{1:t-1})P(X_{1:t-1}) \\&= P(E_t|X_t)P(E_{1:t-1}|X_{1:t})P(X_t|X_{1:t-1})P(X_{t-1}|X_{1:t-2})P(X_{1:t-2}) \\&= \prod_{i=1}^t P(E_i|X_i) \times \prod_{i=1}^t P(X_i|X_{i-1}) \\&= \prod_{i=1}^t [P(E_i|X_i)P(X_i|X_{i-1})]\end{aligned}$$

Is First Order Markov Process Accurate?

- Sometimes true
 - Consider a random walk along x – axis, where you go left/right based on a coin flip.
 - Position at time step t only depends on position at time step $t - 1$.
- Sometimes not
 - For example, in our rain example, probability of raining at time step t may depend on several previous rainy days $t - 1, t - 2, \dots$

Improve First Order Markov Process?

- Solutions

- Increase the order of the Markov process: $P(X_t | X_{t-1}, X_{t-2})$
- Incorporate more state variables:

Temp_t, Humidity_t, Pressure_t, Session_t, etc.

Inference in First Order Markov Process

- **Filtering query:** Compute probability distribution of current state given all observations to date.
 - $P(X_t|e_{1:t})$
 - Compute probability of raining (and not raining also!) today, given all umbrella observations taken so far
 - *Note the use capital and small letters: Capitals specify random variable and small letters specify values of random values.*
 - Required for decision making at current state

Inference in First Order Markov Process

- **Prediction query:** Compute probability distribution of a future state given all observations to date.
 - $P(X_{t+k} | e_{1:t})$
 - Compute probability of raining three days from now, given all umbrella observations taken so far
 - Required for decision making about future action

Inference in First Order Markov Process

- **Smoothing query:** Compute probability distribution of a past state given all observations to date.
 - $P(X_k | e_{1:t}), 0 \leq k < t$
 - Compute probability of raining last Wednesday, given all umbrella observations taken so far
 - Smoothing provides a better estimate than what was made before

Inference in First Order Markov Process

- **Most likely explanation query:** Given a sequence of observation, what is the most likely state sequence that have generated the observation sequence?
 - $P(X_{1:t}|e_{1:t})$
 - Umbrella was observed on first three days and absent on fourth, the most likely state sequence could be it rained first three days and did not rain on fourth.
 - **Speech recognition:** What is the sequence of words given a sequence of sounds?

Filtering

- Compute probability distribution of current state X_{t+1} given observation sequence $e_{1:t+1}$
- Agent maintains the probability distribution of current state X_t at time step t .
- As new evidence e_{t+1} comes up, agent updates its estimation of current state probabilities $P(X_{t+1})$

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}, \mathbf{e}_{t+1}) \quad (\text{dividing up the evidence}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{using Bayes' rule}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{by the sensor Markov assumption}).\end{aligned}$$

The sensor Markov assumption states that the current observation e_{t+1} , depends only on the current state X_{t+1} , and not on previous states or observations.

Filtering

- α : a is a normalizing constant to make probabilities sum up to 1

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}, \mathbf{e}_{t+1}) \quad (\text{dividing up the evidence}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{using Bayes' rule}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{by the sensor Markov assumption}).\end{aligned}$$

Filtering

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}, \mathbf{e}_{t+1}) \quad (\text{dividing up the evidence}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}, \mathbf{e}_{1:t}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{using Bayes' rule}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t}) \quad (\text{by the sensor Markov assumption}).\end{aligned}$$

- How to calculate $P(X_{t+1} | e_{1:t})$?

- Marginalize over X_t : $P(X_{t+1}) = \sum_{x_t} P(X_{t+1}, x_t) = \sum_{x_t} P(X_{t+1} | x_t) P(x_t)$

Marginalization in probability refers to the process of summing out a variable to obtain the probability of another variable.

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t, \mathbf{e}_{1:t}) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \quad (\text{Markov assumption}).\end{aligned}$$

Filtering

$$\begin{aligned}\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t, \mathbf{e}_{1:t}) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \quad (\text{Markov assumption}).\end{aligned}$$

- $P(e_{t+1}|X_{t+1})$ comes from observation/sensor model [given]
- $P(X_{t+1}|x_t)$ comes from the transition model [given]
- $P(x_t|e_{1:t})$ is the probability distribution of states at time step t
 - This part is recurrence and can be computed recursively or iteratively [using dynamic programming approach]

Filtering

$$\begin{aligned} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{e}_{1:t+1}) &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t, \mathbf{e}_{1:t}) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \\ &= \alpha \mathbf{P}(\mathbf{e}_{t+1} \mid \mathbf{X}_{t+1}) \sum_{\mathbf{x}_t} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t) P(\mathbf{x}_t \mid \mathbf{e}_{1:t}) \quad (\text{Markov assumption}). \end{aligned}$$

- Let, $P(X_t | e_{1:t}) = \mathbf{f}_t$ [$\mathbf{f}_t$ is a vector/array of probabilities]
 - $P(X_t = x_i | e_{1:t}) = \mathbf{f}_t[i]$ [$\mathbf{f}_t[i]$ is a single probability value]
- Hence, $\mathbf{f}_t[i] = P(X_t = x_i | e_{1:t})$
$$= \alpha P(e_t | X_t = x_i) \sum_j P(X_t = x_i | X_{t-1} = x_j) P(X_{t-1} = x_j | e_{1:t-1})$$
$$= \alpha \times (o_{il}) \times \sum_j (a_{ji}) (\mathbf{f}_{t-1}[j]) \quad [\text{assume } e_t = e_l \text{ an output value}]$$

Filtering: Forward Algorithm

- $\mathbf{f}_t[i] = \alpha \times (o_{il}) \times \sum_j (a_{ji})(\mathbf{f}_{t-1}[j])$
- $\mathbf{f}_t$ is known as forward probabilities
- How to compute forward probabilities up to time step t ?
 - Start from $t = 1$ and compute $\mathbf{f}_1$ [base condition]
 - Compute going forward in time up to $\mathbf{f}_t$ using the recurrence
 - The algorithm is known as forward algorithm.

Filtering: Forward Algorithm

- $\mathbf{f}_t[i] = \alpha \times (o_{ik}) \times \sum_j (a_{ji})(\mathbf{f}_{t-1}[j])$
- $\mathbf{f}_t$ is known as forward probabilities
- How to compute compute $\mathbf{f}_1$ [base condition]?
 - $\mathbf{f}_1[i] = P(X_1 = x_i | e_1)$
 $= \alpha P(e_1 | X_1 = x_i) P(X_1 = x_i)$
 $= \alpha \times o_{il} \times \pi_i$ [assume $e_1 = e_l$, π_i is the initial (prior) probability of state x_i at start]

Filtering: Example

- Compute $P(R_2|u_{1:2})$
- Day 1: $P(R_1|u_1) = \alpha P(u_1|R_1)P(R_1)$
 - $P(R_1)$ is the prior probability distribution of initial state [at time $t = 1$]
 - If both states are equally likely from START, $P(R_1) = \langle 0.5, 0.5 \rangle$
 - $P(R_1|u_1)$ can now be calculated as:

$$\begin{aligned}\mathbf{P}(R_1 | u_1) &= \alpha \mathbf{P}(u_1 | R_1) \mathbf{P}(R_1) = \alpha \langle 0.9, 0.2 \rangle \langle 0.5, 0.5 \rangle \\ &= \alpha \langle 0.45, 0.1 \rangle \approx \langle 0.818, 0.182 \rangle .\end{aligned}$$

Filtering: Example

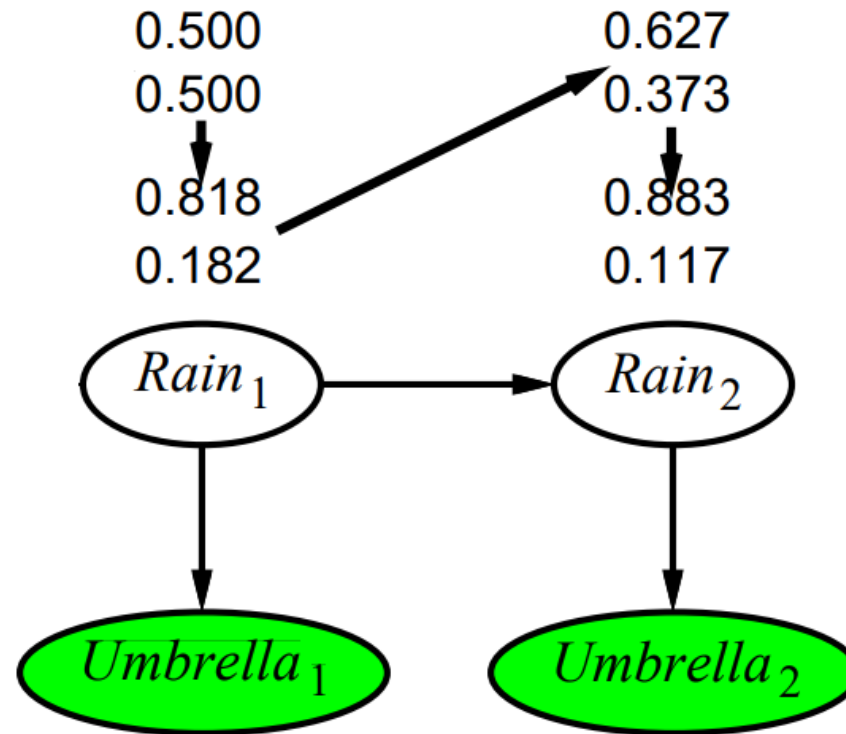
- Day 2: $P(R_2|u_{1:2}) = \alpha P(u_2|R_2)P(R_2|u_1) = \alpha P(u_1|R_1) \sum_{r_1} P(R_1|r_1)P(r_1|u_1)$
 - Can be calculated as:

$$\begin{aligned}\mathbf{P}(R_2 | u_1) &= \sum_{r_1} \mathbf{P}(R_2 | r_1) P(r_1 | u_1) \\ &= \langle 0.7, 0.3 \rangle \times 0.818 + \langle 0.3, 0.7 \rangle \times 0.182 \approx \langle 0.627, 0.373 \rangle\end{aligned}$$

$$\begin{aligned}\mathbf{P}(R_2 | u_1, u_2) &= \alpha \mathbf{P}(u_2 | R_2) \mathbf{P}(R_2 | u_1) = \alpha \langle 0.9, 0.2 \rangle \langle 0.627, 0.373 \rangle \\ &= \alpha \langle 0.565, 0.075 \rangle \approx \langle 0.883, 0.117 \rangle .\end{aligned}$$

Filtering: Example

- Probability of rain increases at day 2 from day 1 [why?]



Prediction

- Compute probability distribution of a future state: $P(X_{t+k}|e_{1:t})$
- Can be computed using filtering:
 - First compute $P(X_t|e_{1:t})$ [*forward algorithm*]
 - Then compute as: $P(X_{t+1}|e_{1:t}) = \sum_{x_t} P(X_{t+1}|x_t)P(x_t|e_{1:t})$.
 - Similarly, compute $P(X_{t+2}|e_{1:t}), \dots, P(X_{t+k}|e_{1:t})$
- Recursive/dynamic programming algorithm:

$$\mathbf{P}(\mathbf{X}_{t+k+1} | \mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+k}} \mathbf{P}(\mathbf{X}_{t+k+1} | \mathbf{x}_{t+k}) P(\mathbf{x}_{t+k} | \mathbf{e}_{1:t}) .$$

Prediction: Don't Go Too Much Ahead

- Recursive/dynamic programming algorithm:

$$\mathbf{P}(\mathbf{X}_{t+k+1} \mid \mathbf{e}_{1:t}) = \sum_{\mathbf{x}_{t+k}} \mathbf{P}(\mathbf{X}_{t+k+1} \mid \mathbf{x}_{t+k}) P(\mathbf{x}_{t+k} \mid \mathbf{e}_{1:t}) .$$

- Predicting too much ahead may be useless
 - $P(X_{t+k+1} | e_{1:t})$ will become fixed (stationary distribution of the Markov Process) after some time steps k
 - The time taken to reach the fixed point is known as Mixing Time.
- The more uncertainty in the transition model, the shorter will be the mixing time and the more future is obscured!

Likelihood of Evidence Sequence

- What is the likelihood of evidence sequence $e_{1:t}$?

- Compute as

- $P(e_{1:t}) = \sum_{x_t} P(e_{1:t}, x_t)$

- $P(e_{1:t}, X_t)$ can be calculated recursively or using dynamic programming:

$$\begin{aligned} P(e_{1:t}, X_t) &= P(e_{1:t-1}, e_t, X_t) = P(e_t | X_t, e_{1:t-1}) P(X_t, e_{1:t-1}) \\ &= P(e_t | X_t) \sum_{x_{t-1}} P(X_t, x_{t-1}, e_{1:t-1}) \quad [\text{Markov assumption}] \\ &= P(e_t | X_t) \sum_{x_{t-1}} P(X_t | x_{t-1}) P(e_{1:t-1}, x_{t-1}) \end{aligned}$$

- $P(x_{t-1}, e_{1:t-1})$ can be computed recursively [using dynamic programming]
 - This is similar to the forward algorithm [described earlier]

Likelihood of Evidence Sequence

- Similar recurrence a $P(X_t|e_{1:t})$ [without normalization constant α]:

$$P(e_{1:t}, X_t) = P(e_t|X_t) \sum_{x_{t-1}} P(X_t|x_{t-1})P(e_{1:t-1}, x_{t-1})$$

- Recurrence: $\mathbf{f}_t[i] = (o_{il}) \times \sum_j (a_{ji})(\mathbf{f}_{t-1}[j])$
 - $\mathbf{f}_t[i] = P(e_{1:t}, X_t = x_i), x_t = x_i, x_{t-1} = x_j, e_t = e_l$
- $P(e_{1:t}, X_t)$ is also known as forward probabilities

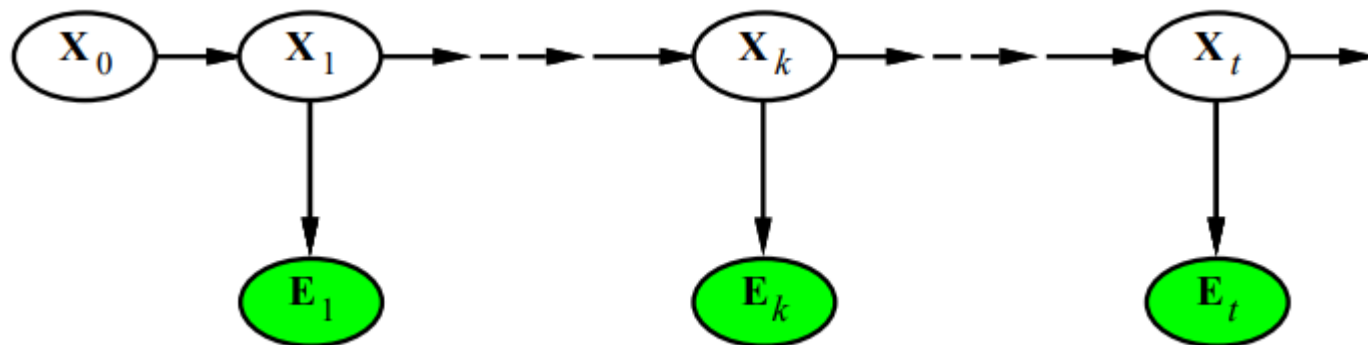
Chapter 15 (AIAMA)

Probabilistic Reasoning Over Time-02

Sukarna Barua
Associate Professor, CSE, BUET

Smoothing

- Compute the probability distribution over past states.
- Compute $P(X_k | e_{1:t})$ where $0 \leq k < t$



Smoothing

- Compute $P(X_k | e_{1:t})$ where $0 \leq k < t$

$$\begin{aligned} \mathbf{P}(X_k | \mathbf{e}_{1:t}) &= \mathbf{P}(X_k | \mathbf{e}_{1:k}, \mathbf{e}_{k+1:t}) \\ &= \alpha \mathbf{P}(X_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | X_k, \mathbf{e}_{1:k}) \quad (\text{using Bayes' rule}) \\ &= \alpha \mathbf{P}(X_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | X_k) \quad (\text{using conditional independence}) \end{aligned}$$

- Remember bayes expansion: $P(a|b, c) = \frac{P(b|a, c)P(a|c)}{P(b|c)}$
- Here, $P(X_k | e_{k+1:t}, e_{1:k}) = \frac{P(e_{k+1:t} | X_k, e_{1:k})P(X_k | e_{1:k})}{P(e_{k+1:t} | e_{1:k})}$
- $P(e_{k+1:t} | e_{1:k})$ is a fixed term α [can be obtained later as probabilities sum to 1]

Smoothing

- Compute $P(X_k | e_{1:t})$ where $0 \leq k < t$

$$\begin{aligned} \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:t}) &= \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}, \mathbf{e}_{k+1:t}) \\ &= \alpha \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k, \mathbf{e}_{1:k}) \quad (\text{using Bayes' rule}) \\ &= \alpha \mathbf{P}(\mathbf{X}_k | \mathbf{e}_{1:k}) \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k) \quad (\text{using conditional independence}) \end{aligned}$$

- Hence, $P(X_k = x_i | e_{1:t}) = \alpha \mathbf{f}_k[i] \times P(e_{k+1:t} | X_k = x_i)$
 - Where $\mathbf{f}_k$ denotes forward probabilities [*filtering problem*]
 - Note that $\mathbf{f}_k$ is a vector [$\mathbf{f}_k[i] = P(X_k = x_i | e_{1:k})$]

Smoothing

- Now, how to compute $P(e_{k+1:t} | X_k)$?

$$\begin{aligned} \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k) &= \sum_{\mathbf{x}_{k+1}} \mathbf{P}(\mathbf{e}_{k+1:t} | \mathbf{X}_k, \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) \quad (\text{conditioning on } \mathbf{X}_{k+1}) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) \quad (\text{by conditional independence}) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1}, \mathbf{e}_{k+2:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k) \\ &= \sum_{\mathbf{x}_{k+1}} P(\mathbf{e}_{k+1} | \mathbf{x}_{k+1}) P(\mathbf{e}_{k+2:t} | \mathbf{x}_{k+1}) \mathbf{P}(\mathbf{x}_{k+1} | \mathbf{X}_k), \end{aligned} \tag{15.9}$$

- $P(e_{k+1} | x_{k+1})$ and $P(x_{k+1} | x_k)$ comes from the models [transition and sensor]
- $P(e_{k+2:t} | x_{k+1})$ can be obtained recursively or iteratively [*using dynamic programming*]

Smoothing

- **Recurrence relation for $P(e_{k+1:t}|X_k)$:**
- Let $P(e_{k+1:t}|X_k) = \mathbf{b}_k$ [$\mathbf{b}_k$ is a vector/array of probabilities]

- From our previous derivation:

$$\begin{aligned} - \quad \mathbf{b}_k[i] &= P(e_{k+1:t}|X_k = x_i) \\ &= \sum_{x_j} P(e_{k+1}|X_{k+1} = x_j)P(X_{k+1} = x_j |X_k = x_i) \mathbf{b}_{k+1}[j] \\ &= \sum_{x_j} (o_{j,l})(a_{ij})(\mathbf{b}_{k+1}[j]) \end{aligned}$$

[assume $e_{k+1} = e_l$ an specific output value at time step $k + 1$]

Smoothing

- **Recurrence relation for $P(e_{k+1:t}|X_k)$:**

- $\mathbf{b}_k[i] = \sum_j (o_{jl})(a_{ij})(\mathbf{b}_{k+1}[j]) \quad [e_l = e_{k+1}]$

- What is the base condition?

- To find the base condition, put $k = t$ [for the last/current state]

- $\mathbf{b}_t[j] = P(e_{t+1:t}|X_t = x_j) = 1$ for all j [Why?]

- Probability of occurring an empty sequence $e_{t+1:t}$ is 1

Smoothing

- **Recurrence relation for $P(e_{k+1:t}|X_k)$:**

$$\mathbf{b}_k[i] = \sum_j (o_{jl})(a_{ij})(\mathbf{b}_{k+1}[j])$$

- $\mathbf{b}_k = P(e_{k+1:t}|X_k)$ is known as backward probabilities
 - The algorithm for computing $\mathbf{b}_k$ starts from the t -th state [$k = t$] and go backward up to $k = 1$ [*Fill the dp tables for $k = t$ to 1*]
 - The algorithm is known as backward algorithm [*in contrast to forward algorithm*]

Smoothing

- **Finally smoothed probability** $P(X_k = x_i | e_{1:t})$:

$$P(X_k = x_i | e_{1:t}) = \alpha \times \mathbf{f}_k[i] \times \mathbf{b}_k[i]$$

- In vector form:

$$P(X_k | e_{1:t}) = \alpha \times \mathbf{f}_k \times \mathbf{b}_k \quad [\textit{point-wise multiplication}]$$

- Again α is normalizing constant.

Smoothing: Example

- Compute $P(R_1|u_1, u_2)$ [Probability of rain at time $t = 1$ given umbrella observations at time $t = 1$ and $t = 2$]
- As per our previous formula: $\mathbf{P}(R_1 | u_1, u_2) = \alpha \mathbf{P}(R_1 | u_1) \mathbf{P}(u_2 | R_1)$.
- First term: $P(R_1|u_1) = \langle 0.818, 0.182 \rangle$ [We already know from filtering example]
- Second term: $P(u_2|R_1)$ needs recursive expansion [previous slide]

$$\begin{aligned}\mathbf{P}(u_2 | R_1) &= \sum_{r_2} P(u_2 | r_2) P(r_2) \mathbf{P}(r_2 | R_1) \\ &= (0.9 \times 1 \times \langle 0.7, 0.3 \rangle) + (0.2 \times 1 \times \langle 0.3, 0.7 \rangle) = \langle 0.69, 0.41 \rangle .\end{aligned}$$

Smoothing: Example

- Second term $P(u_2 | R_1)$ needs recursive expansion [previous slides]

$$\begin{aligned}\mathbf{P}(u_2 | R_1) &= \sum_{r_2} P(u_2 | r_2) P(r_2 | R_1) \mathbf{P}(r_2 | R_1) \\ &= (0.9 \times 1 \times \langle 0.7, 0.3 \rangle) + (0.2 \times 1 \times \langle 0.3, 0.7 \rangle) = \langle 0.69, 0.41 \rangle .\end{aligned}$$

- Finally, compute:

$$\mathbf{P}(R_1 | u_1, u_2) = \alpha \mathbf{P}(R_1 | u_1) \mathbf{P}(u_2 | R_1) .$$

$$\mathbf{P}(R_1 | u_1, u_2) = \alpha \langle 0.818, 0.182 \rangle \times \langle 0.69, 0.41 \rangle \approx \langle 0.883, 0.117 \rangle .$$

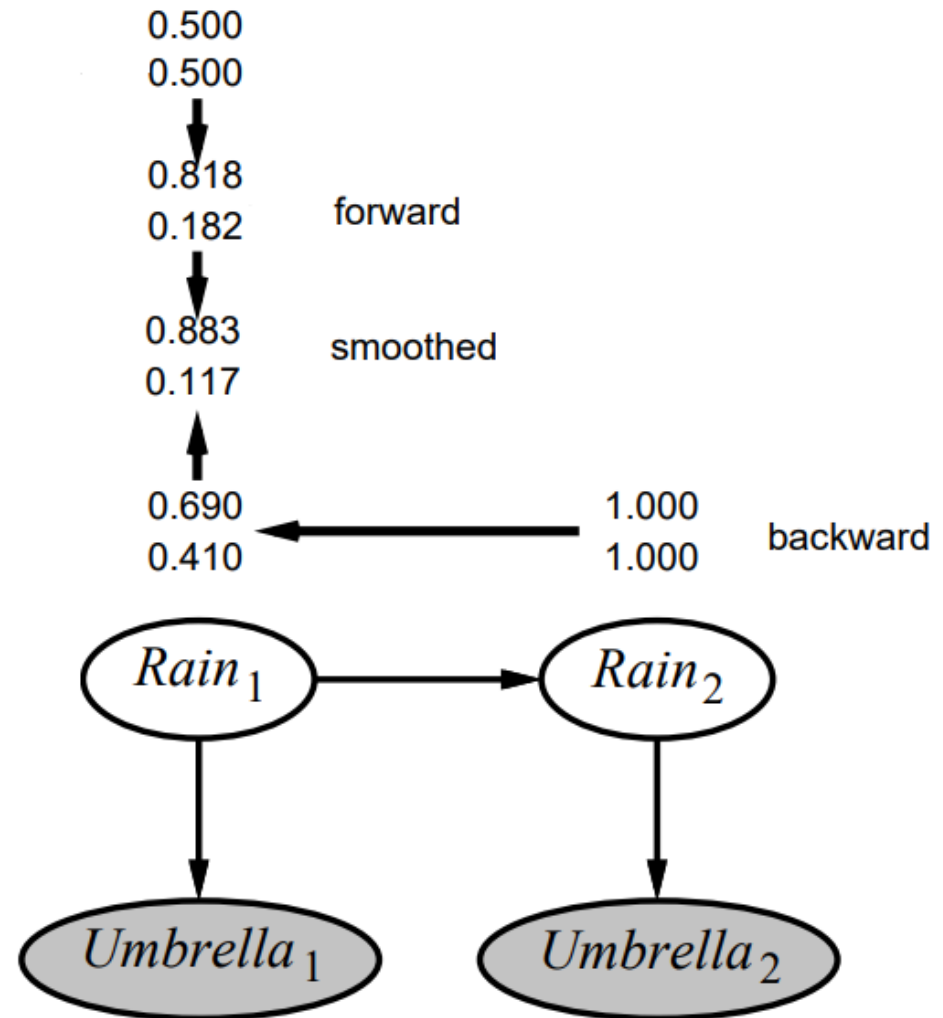
Smoothing: Example

- Smoothed estimate:

$$\mathbf{P}(R_1 | u_1, u_2) = \langle 0.883, 0.117 \rangle$$

- Filtered estimate:

$$\mathbf{P}(R_1 | u_1) = \langle 0.818, 0.182 \rangle$$



Smoothing: Example

- Smoothed estimate: $\mathbf{P}(R_1 | u_1, u_2) = \langle 0.883, 0.117 \rangle$
- Filtered estimate: $\mathbf{P}(R_1 | u_1) = \langle 0.818, 0.182 \rangle$
- Observation: Smoothed estimate for rain on day 1 is higher than the filtered estimate (0.818) [Why?]
 - This is because the umbrella on day 2 makes it more likely to have rained on day 2 which in turn, because rain tends to persist, that makes it more likely to have rained on day 1.

Smoothing: Algorithm Pseudocode

- Forward and backward algorithm can be combined to compute posterior probabilities in linear time [*remember to initialize **b***]

```
function FORWARD-BACKWARD(ev, prior) returns a vector of probability distributions
  inputs: ev, a vector of evidence values for steps  $1, \dots, t$ 
           prior, the prior distribution on the initial state,  $\mathbf{P}(\mathbf{X}_0)$ 
  local variables: fv, a vector of forward messages for steps  $0, \dots, t$ 
                     b, a representation of the backward message, initially all 1s
                     sv, a vector of smoothed estimates for steps  $1, \dots, t$ 

  fv[0]  $\leftarrow$  prior
  for  $i = 1$  to  $t$  do
    fv[ $i$ ]  $\leftarrow$  FORWARD(fv[ $i - 1$ ], ev[ $i$ ])
  for  $i = t$  downto 1 do
    sv[ $i$ ]  $\leftarrow$  NORMALIZE(fv[ $i$ ]  $\times$  b)
    b  $\leftarrow$  BACKWARD(b, ev[ $i$ ])
  return sv
```

Most Likely Explanation

- Suppose that [true, true, false, true, true] is the umbrella sequence for the security guard's first five days on the job.
- What is the weather (state) sequence most likely to explain this?
- There are 2^5 possible weather sequences. Is there a way to find the most likely one?

Most Likely Explanation: Naïve Approach

- Compute the probability distribution $P(X_{1:t}|e_{1:t})$.
 - $P(X_{1:t}|e_{1:t}) = \alpha P(e_{1:t}|X_{1:t})P(X_{1:t})$ [Bayes theorem]
 $= \alpha \prod_{i=1}^t P(e_i|X_i) \prod_{i=1}^t P(X_i|X_{i-1}) = \alpha (\prod_{i=1}^t o_{i,i}) (\prod_{i=1}^t a_{i,i-1})$
 - Assume $a_{1,o} = P(X_1|X_0) = P(X_1)$ for notational convenience
 - $P(X_1)$ is the probability distribution of states at time step $t = 1$
- After computing the probability distribution $P(X_{1:t}|e_{1:t})$
 - We know the probability of each state sequence $P(x_{1:t}|e_{1:t})$
 - We select the most probably state sequence as

$$\operatorname{argmax}_{x_{1:t}} P(x_{1:t}|e_{1:t})$$

Most Likely Explanation: Naïve Approach

- We select the most probably state sequence as:

$$\operatorname{argmax}_{x_{1:t}} P(x_{1:t}|e_{1:t})$$

- What is the problem with the naïve approach?
 - Number of state sequence is exponential
 - If number of states is N , then number of state sequences is N^t
 - Computing $P(x_{1:t}|e_{1:t})$ for N^t states requires exponential running time
 - Not feasible in real time
 - What can we do?
 - A better approach can be derived using dynamic programming

Most Likely Explanation

- Note that $\operatorname{argmax}_{x_{1:t}} P(x_{1:t}|e_{1:t}) = \operatorname{argmax}_{x_{1:t}} P(x_{1:t}, e_{1:t})$ [Why?]
- Consider the probability: $\beta_t[i] = \max_{x_{1:t-1}} P(X_t = x_i, x_{1:t-1}, e_{1:t})$
 - $\beta_t[i]$ is the probability of the most likely state sequence that produce observation sequences $e_{1:t}$ and ends at a specific state x_i at time step t .
 - We will show that instead of maximizing over all sequence of previous states, it is enough to maximize over only previous state [*we will derive a recurrence relation over the previous state*]

Most Likely Explanation

- **Recurrence relation for computing $\beta_t[i]$:** we can compute $\beta_t[i]$ as follows:
 - For all possible states x_j at time step $t - 1$:
 - Compute the probability of most likely state sequence that produce observation sequence $e_{1:t-1}$ and ends at $X_{t-1} = x_j$ (this probability is $\beta_{t-1}[j]$)
 - Move from state x_j to x_i [with transition probability a_{ji}]
 - Produce observation e_t at state x_i [with emission probability o_{il} assuming $e_t = e_l$]
 - As we don't know the previous state x_j leading to the most likely sequence, we just take the maximum over all possible previous states x_j :

$$\beta_t[i] = \max_j [\beta_{t-1}[j] P(x_i | x_j) P(e_t | x_i)] = o_{il} \max_j \{a_{ji} \beta_{t-1}[j]\}$$

Most Likely Explanation

- Derivation of recurrence relation for $\beta_t[i]$:

$$\begin{aligned}\beta_t[i] &= \max_{x_{1:t-1}} P(X_t = x_t, x_{1:t-1}, e_{1:t}) = \max_{x_{1:t-1}} P(X_t = x_t, x_{1:t-1}, e_{1:t-1}, e_t) \\&= \max_{x_{1:t-1}} P(e_t | X_t = x_t) P(X_t = x_t, x_{1:t-1}, e_{1:t-1}) \\&= P(e_t | X_t = x_t) \max_{x_{1:t-1}} P(X_t = x_t | x_{t-1}) P(x_{1:t-1}, e_{1:t-1}) \\&= P(e_t | X_t = x_t) \max_{X_{t-1}=x_j} \{ P(X_t = x_t | X_{t-1} = x_j) \max_{x_{1:t-2}} P(x_{t-1} = x_j, x_{1:t-2}, e_{1:t-2}) \} \\&= P(e_t | X_t = x_t) \max_j P(X_t = x_t | X_{t-1} = x_j) \beta_{t-1}[j] \\&= o_{it} \max_j (a_{ji} \beta_{t-1}[j])\end{aligned}$$

Most Likely Explanation

- Finally obtain the best possible state sequence:
 - $\beta_t[i]$ is the most likely state sequence ending at x_i at time step t
 - We don't know what is the end state for the most likely state sequence, hence we just take the maximum over all $\beta_t[i]$ s:

$$\operatorname{argmax}_{x_{1:t}} P(x_{1:t}, e_{1:t}) = \operatorname{argmax}_i \beta_t[i]$$

- This algorithm is known as Viterbi algorithm
- Can be implemented using recursion or dynamic programming approach

Chapter 17 (AIAMA)

Making Complex Decisions

Sukarna Barua
Associate Professor, CSE, BUET

Sequential Decision Problems

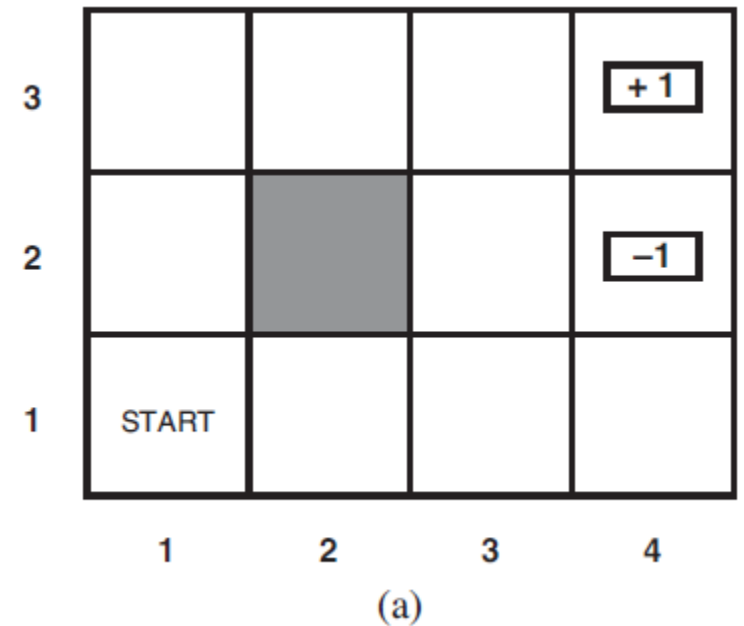
In a sequential decision problem:

- Utility do not depend on a single decision/action.
 - Example: You have two choices in a game: First choice gives you 10k with probability $\frac{1}{2}$ and 0k otherwise. Second choice gives you 30k with probability $\frac{1}{4}$ and 5k otherwise. Which action should a rational agent choose? It is simple to choose this single action!
- Utility depends on a sequence of agent's decisions/actions.
 - What an agent should do at any time depends on what it will do in the future.
 - What an agent does in the future depends on what it did before.

Sequential Decision Problems

Consider an agent's exploration in a 4×3 grid environment:

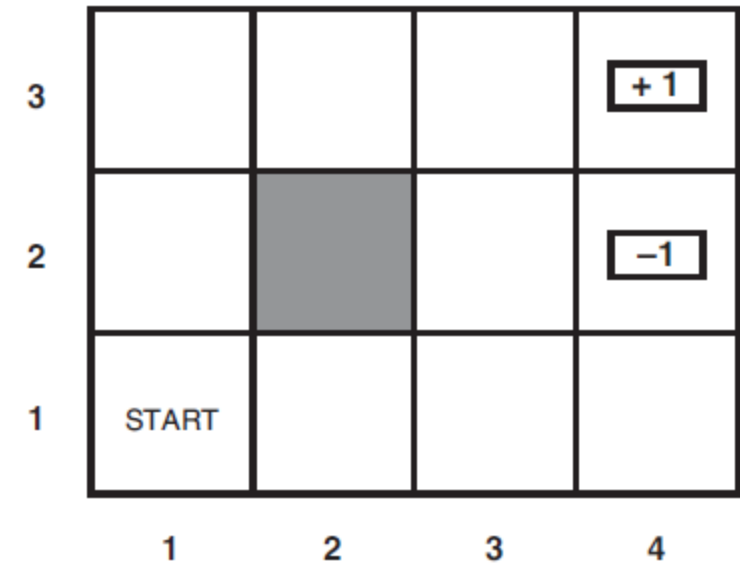
- Agent begins in the START state
- Agent chooses an action in each state
 - **Actions:** UP, LEFT, DOWN, RIGHT
- Exploration ends when agent reaches one of the goal states (+1 or -1)



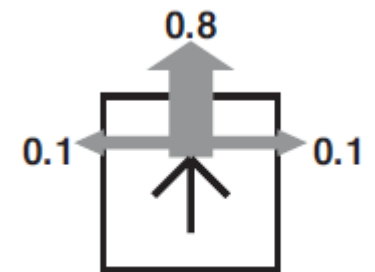
Sequential Decision Problems

Actions of agent:

- Agent chooses an action in each state
 - Actions: UP, LEFT, DOWN, RIGHT
- Action effect is not deterministic rather stochastic
 - Each action takes agent in the right direction with probability 0.8
 - With 0.1 probability, agent can move to the right angles of the intended direction
 - If agent hits a wall, it bumps and stays in the same location



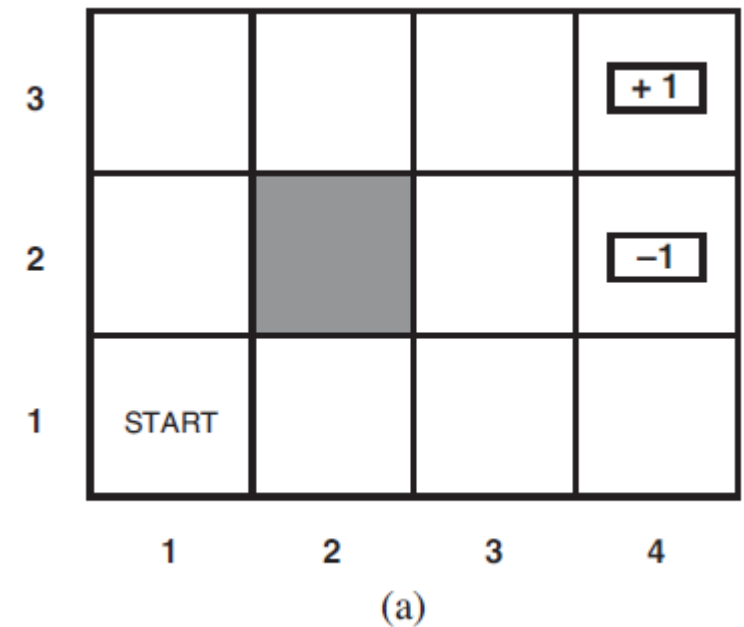
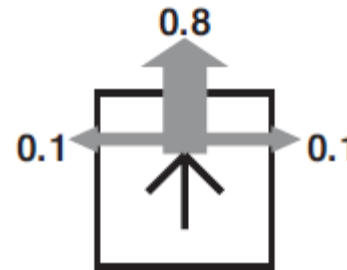
(a)



Sequential Decision Problems

Actions of agent: Example

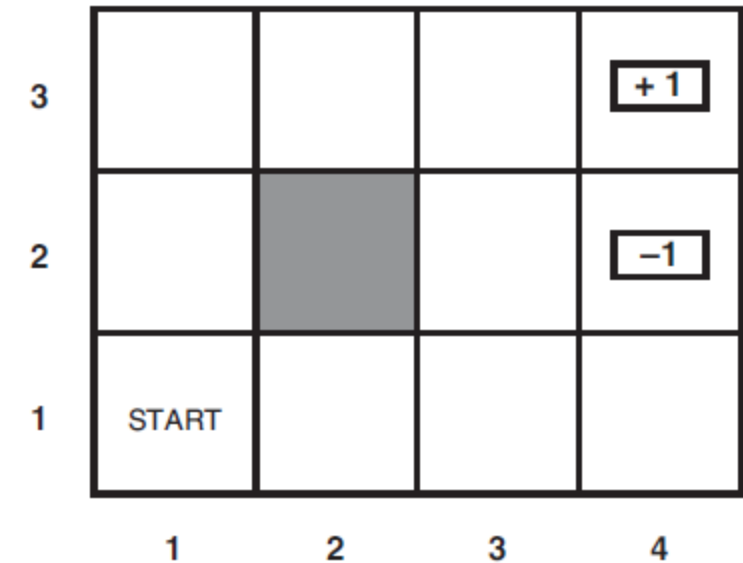
- Agent chooses action UP in (1,1)
 - With 0.8 prob, agent moves to (1,2)
 - With 0.1 prob, agent moves to (2,1)
 - With 0.1 probability, moves to left, bumps to wall and stays at (1,1)



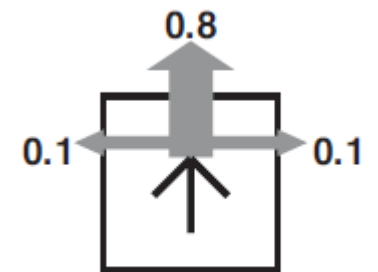
Sequential Decision Problems

What if action effect was deterministic?

- The problem turns into a simple search problem:
 - Find the sequence of moves leading to goal.
 - DFS/BFS can easily find this.
- In sequential decision problem:
 - Action effect is not deterministic.
 - Simple search do not work [Why? We need action for for every state instead of a shortest path to goal]

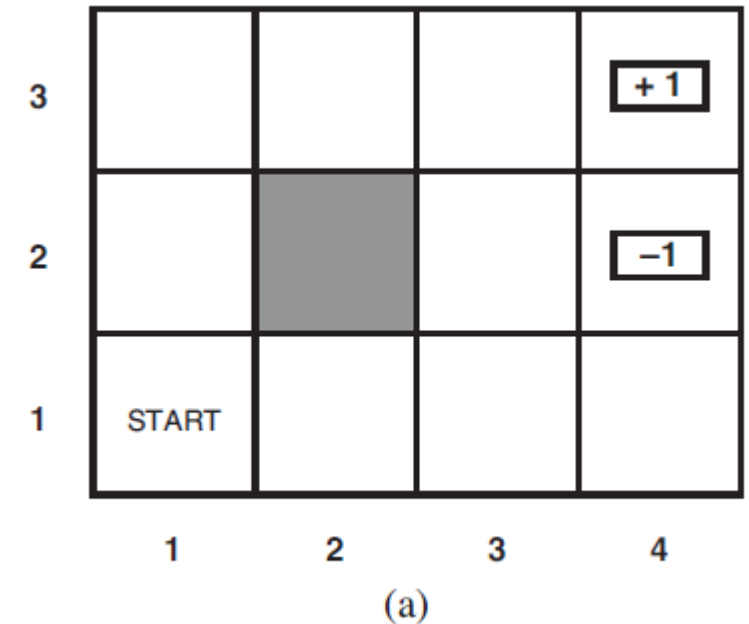


(a)



Sequential Decision Problems

- Agent cannot reach goal by following the action sequence: UP, UP, RIGHT, RIGHT, RIGHT. [Why?]
 - Because action effects are not deterministic
- What is the probability that agent reaches +1 by following the path given above as action sequence?
 - Answer: $0.8^5 = 0.32768$

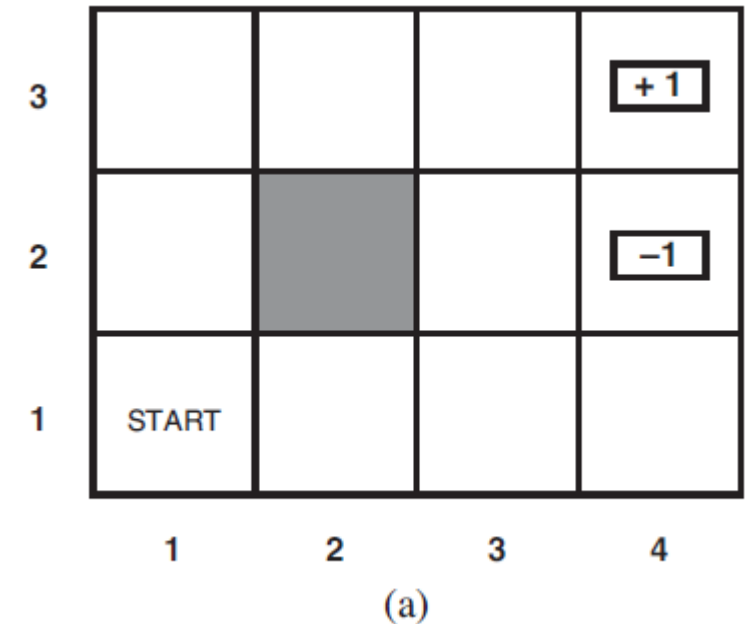


Sequential Decision Problems

Rewards from the environment:

Agent receives a reward from the environment

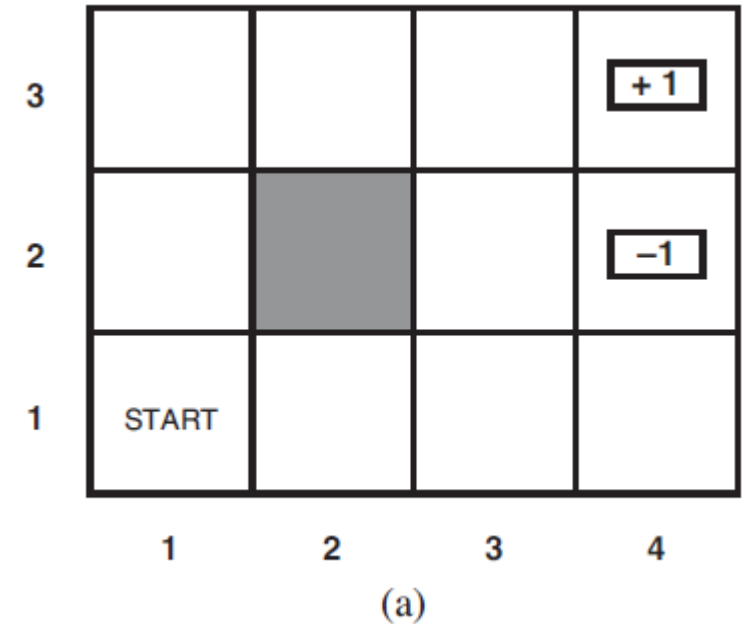
- For state s , reward is $R(s)$
 - $R(s) = -1.0$ and $+1.0$ for terminal states
 - $R(s) = -0.04$ in all other states
- The utility of a sequence of steps is the sum of all rewards.
 - Say, agent reaches $+1$ state after 10 steps
 - total utility $= 10 \times -0.04 + 1.0 = 0.6$



Sequential Decision Problems

Utility from the environment:

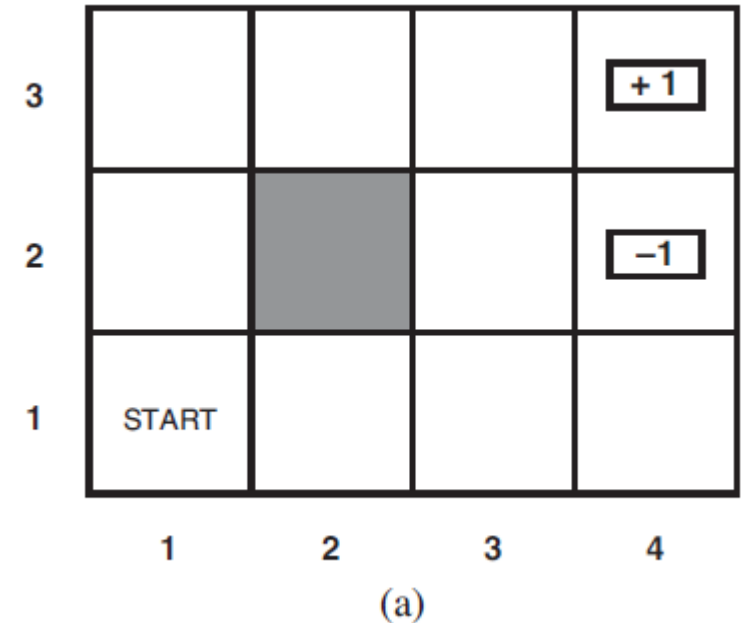
- Agent tries to reach +1 as quickly as possible
 - Otherwise, agent gets negative reward and utility decrease
 - Consider, agent reaches +1 with 10 steps vs 5 steps.
 - For 10 steps, total utility received = 0.6
 - For 5 steps, total utility received = 0.8
- Real utility is gained only from terminal states.
 - However, utility can be decreased/increased by the cost/penalty incurred from visiting each state s to reach the terminal state.



Sequential Decision Problems

Transition model:

- Agent moves a new state by following an action in current state.
 - Modeled by a transition probability $P(s'|s, a)$
 - Can be described by a three-dimensional table specifying all probability values
- The transition model is Markovian
 - $P(s'|s, a)$ depends only on current state s and not on any previous states visited.
 - Example: $P((1,2)|(1,1), UP) = 0.8, P((2,1)|(1,1), UP) = 0.1$, etc.



Markov Decision Process

- **A Markov Decision Process (MDP) is a sequential decision problem where:**
 - Environment is fully observable [*Agent knows where it is*]
 - Environment is stochastic [*Actions do not always leads to the intended state*]
 - Transition model is Markovian
 - Rewards are additive
- **A MDP consists of**
 - A set of states (with initial state s_0)
 - A set of actions $ACTIONS(s)$ in each state s
 - A transition model $P(s'|s, a)$
 - A reward model $R(s)$

Markov Decision Process

- A **Partially Observable Markov Decision Process (POMDP)** is a sequential decision problem where:
 - Environment is partially observable
 - Agent does not necessarily know where it is in
 - Reward and action do not just depend on s but also on how much the agent knows when it is in s
 - More complex than ordinary MDPs. [*We won't study in this course*]

Markov Decision Process: Solution

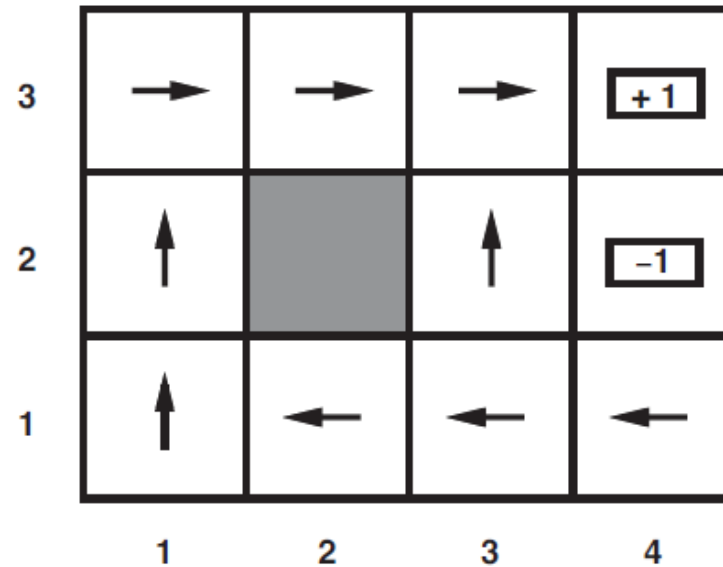
- **A solution to an MDP is called a policy that defines**
 - Which action agent should follow for every state
 - Usually denoted by π or $\pi(s)$.
 - If agent is given to follow a policy, the agent always takes the action given in the policy, no matter what the outcome is.
- **A policy π**
 - Is not an action-planning [*which actions leads to goal, simple search problem*]
 - In fact, an action-sequence is not guaranteed to take to goal.
 - Is a strategy planning [*which action to take at each state, MDP*]

Markov Decision Process: Optimal Policy

- A policy π
 - Does not always takes an agent to the same state [*stochastic action effects*]
 - Each execution of a policy may result in a different state sequence followed by the agent [*due to the stochastic nature of the action effect*].
- What makes a policy π **optimal**?
 - A policy is optimal if it yields the highest expected utility [*not a reward*]
 - Usually denoted by π^*

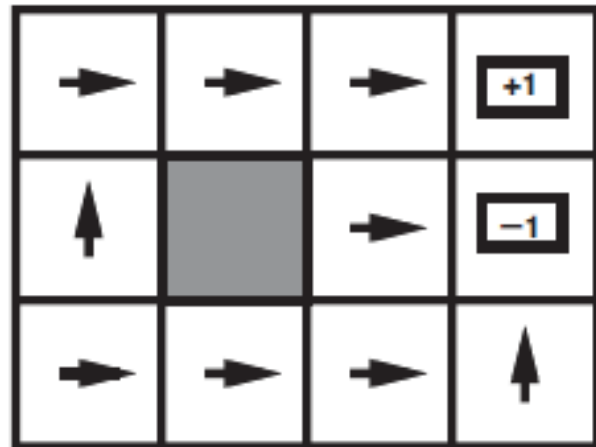
Markov Decision Process: Optimal Policy

- **Example:** An optimal policy for the grid world problem with $R(s) = -0.04$ for nonterminal states.

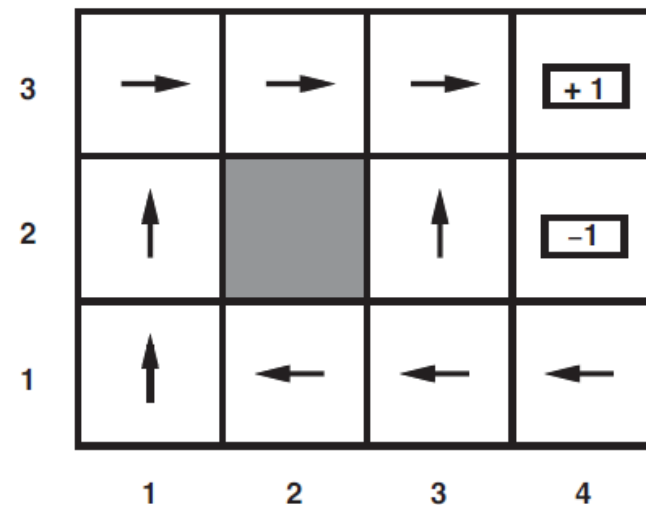


Markov Decision Process: Optimal Policy

- What happens to optimal policy when $R(s) \leq -1.6284$?
 - Life is so painful that the agent heads straight for the nearest exit, even if the exit is worth -1 .
 - Note the action at $(3,2)$ [Agent chooses -1 to quickly exit]

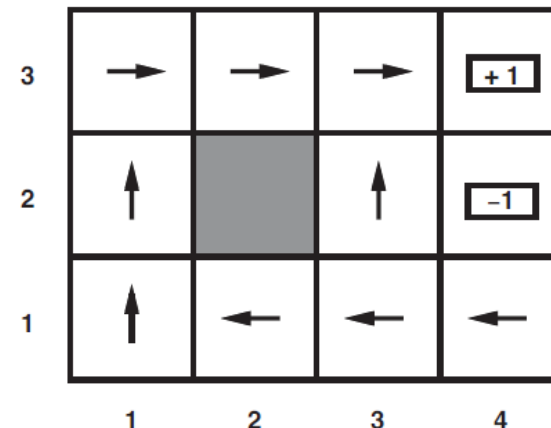
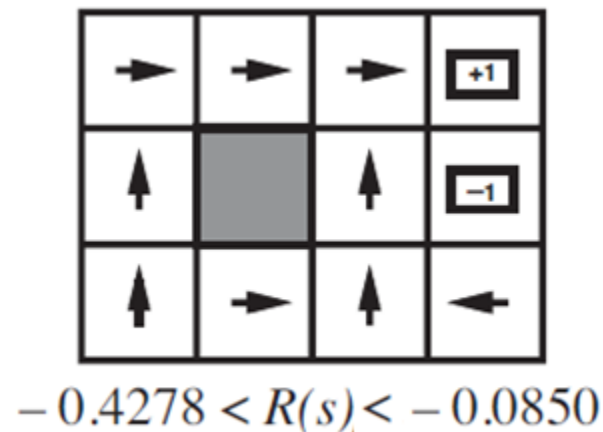


$$R(s) < -1.6284$$



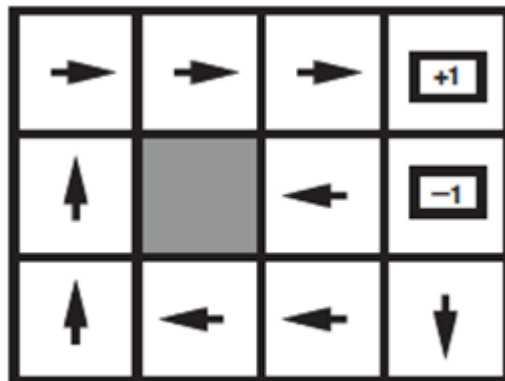
Markov Decision Process: Optimal Policy

- What happens to optimal policy when $-0.4278 < R(s) < -0.0850$?
 - Life is not so painful but unpleasant; the agent takes the shortest route to +1, and is willing to risk falling to -1 by accident
 - Note the action at (3,1) [Agent chooses UP to quickly exit]

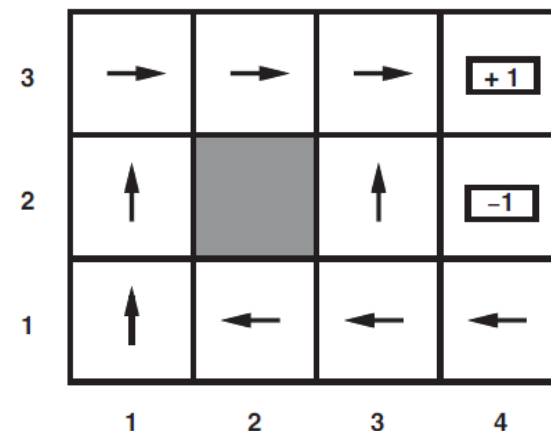


Markov Decision Process: Optimal Policy

- What happens to optimal policy when $-0.0221 < R(s) < 0$?
 - The optimal policy takes *no risks at all*. In (4,1) and (3,2), the agent heads directly away from the -1 state so that it cannot fall in by accident, even though this means banging its head against the wall quite a few times.



$$-0.0221 < R(s) < 0$$



Markov Decision Process: Reward Types

- Assume an agent visits an state sequence $[s_0, s_1, \dots]$. The utility obtained is $U_h([s_0, s_1, \dots])$ can be considered in two ways:
 - **Additive rewards:** $U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) \dots$
 - Already considered in our previous grid world example
 - **Discounted rewards:** $U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$
 - Future rewards are discounted by a factor γ where $0 < \gamma \leq 1$.
 - γ close 1: Agent considers future rewards as highly as current rewards
 - γ close 0: Agent considers distant rewards insignificant

Markov Decision Process: Expected Utility is Bounded

- With discounted rewards, the utility obtained from visiting an infinite sequence of states is still finite as shown below:
 - Assume rewards are bounded by $\pm R_{max}$ and $\gamma < 1$
 - The total utility after infinite number of steps is:

$$U_h = \sum_{i=0}^{\infty} \gamma^i R(s_i) \leq \sum_{i=0}^{\infty} \gamma^i R_{max} = R_{max} \left(\frac{1}{1 - \gamma} \right)$$

Markov Decision Process: Expected Utility of a Policy

- Assume agent starts at initial state s .
- Agent reaches state S_t at time t when executing a policy π [Note S_t is not a deterministic state; it is a random variable. Why?]
- Agent acts in an infinite-horizon [*no limit on the # of steps, but exploration ends as soon as terminal state is reached*]
- The probability distribution over the state sequences $S_1, S_2, \dots$ visited by the agent is determined by:
 - The initial state s
 - The policy π
 - The transition model $P(s'|s, a)$

Markov Decision Process: Expected Utility for a Policy

- The expected utility obtained by executing a given policy is computed as:

$$U^\pi(s) = E[\sum_{t=0}^{\infty} \gamma^t R(S_t)] \text{ where } S_0 = s$$

[We can't just sum $R(S_t)$ as $R(S_t)$ is a random variable having a probability distribution over states, so we need to take the expected value]

- Can we compute $U^\pi(s)$ using the above equation?
 - Not easy!
 - As t increases, computing $R(S_t)$ becomes extremely difficult *[with exponential number of values for the random variable S_t , Why?]*

Markov Decision Process: Optimal Policy

- **Optimal Policy:** The optimal policy is the one that gives the maximum expected utility:

$$\pi_s^* = \operatorname{argmax}_{\pi} U^{\pi}(s)$$

- π_s^* is the optimal policy starting with state s .
 - π_s^* is a policy; hence it recommends an action for every state.
- Optimal policy is independent of start state s . [Why?]
 - Remember policy is like a function specifying an action for every state; hence it does not matter where you start; you always have an action-sequence as per the optimal policy.
- We can simply write the optimal policy as π^* .

Markov Decision Process: Utility Function

- The maximum achievable utility starting from any state s is given by: $U^{\pi^*}(s)$
 - The sum of the discounted rewards if agent executed optimal policy starting from s .
 - We simply express it $U(s)[= U^{\pi^*}(s)]$
 - Note $U(s)$ and $R(s)$ are quite different quantities!
 - Note $U(s)$ and $U^{\pi}(s)$ are two different quantities!
 - $U(s)$ depends on s [Why?]

Markov Decision Process: Utility Function

- Following figure shows the maximum utility $U(s)$ for different states in the grid world problem for $\gamma = 1$ and $R(s) = -0.04$
- Note that states closer to +1 have higher utilities compared to states far from it.
 - Why? [Because starts far away needs more steps to reach +1, thus additive rewards (-0.04 per state) reduces the utility]

3	0.812	0.868	0.918	+ 1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Markov Decision Process: Optimal Action

- Agent can use the utility function $U(s)$ to choose the best action at each state:
 - Choose the action a in state s which gives you maximum utility from the next possible states; such action-sequence defines an optimal policy.

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} U(s') P(s'|s, a)$$

- The above is the optimal policy equation.

Markov Decision Process: Value Iteration

- **Value iteration:** An algorithm to calculate the optimal policy
- Basic idea of the algorithm:
 - Step 1: Calculate utility $U(s)$ for every state s [Remember $U(s)$ is the maximum utility achievable from state s by following optimal policy]
 - Step 2: Calculate optimal action for each state based on calculated $U(s)$ in Step 1 using optimal policy equation [*see previous slide*].
 - Seems chicken and egg problem? [*As in Step 1, we don't have an optimal policy!*]
 - No! $U(s)$ can be calculated without knowing optimal policy!

Markov Decision Process: Bellman Equation

- **Utility function:** Let's define utility function in a different way.
- The optimal utility $U(s)$ of a state s satisfies the following equation:

$$U(s) = R(s) + \gamma \max_a \sum_{s'} P(s'|s, a) U(s')$$

- $U(s)$ = immediate reward + expected discounted utility of next state [*assume agent follows optimal action as per optimal policy*]
 - This is optimal substructure property for the optimal utility!
- This is called **Bellman equation**.

Markov Decision Process: Bellman Equation Example

- Let's verify Bellman Equation for grid world problem.
- Bellman equation for state (1,1) is:

$$U(1, 1) = -0.04 + \gamma \max \begin{cases} 0.8U(1, 2) + 0.1U(2, 1) + 0.1U(1, 1), & (Up) \\ 0.9U(1, 1) + 0.1U(1, 2), & (Left) \\ 0.9U(1, 1) + 0.1U(2, 1), & (Down) \\ 0.8U(2, 1) + 0.1U(1, 2) + 0.1U(1, 1) \end{cases}.$$

- Plugging in numbers from the figure:
 - We can show UP is the best action
 - Choose any other action (i.e., RIGHT) and verify Bellman equation won't be satisfied.

3	0.812	0.868	0.918	+1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Markov Decision Process: Value Iteration Algorithm

- Based on Bellman equations
- Number of states = n , Number of Bellman equations = n
- Number of unknowns = n [$U(s)$ for every state s]
- If equations were linear, we could easily use Gauss Elimination to solve.
 - Bellman equations are non-linear due to the max operation!
 - Hence, easier approaches won't help
- Solution: iterative optimization known as **Value Iteration** algorithm.

Markov Decision Process: Value Iteration Algorithm

- Iterative approach to solve Bellman equations [**Value Iteration Algorithm**]
 - Start with random values for U s [*usual practice: start with all zeros*]
 - Repeat until equilibrium:
 - Calculate right hand side of Bellman equations using current values of U 's and use it to update left-hand side (new updated values for U 's):

$$U_{i+1}(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s, a) U_i(s')$$

[The above step is known as **Bellman update**]

Markov Decision Process: Value Iteration Algorithm

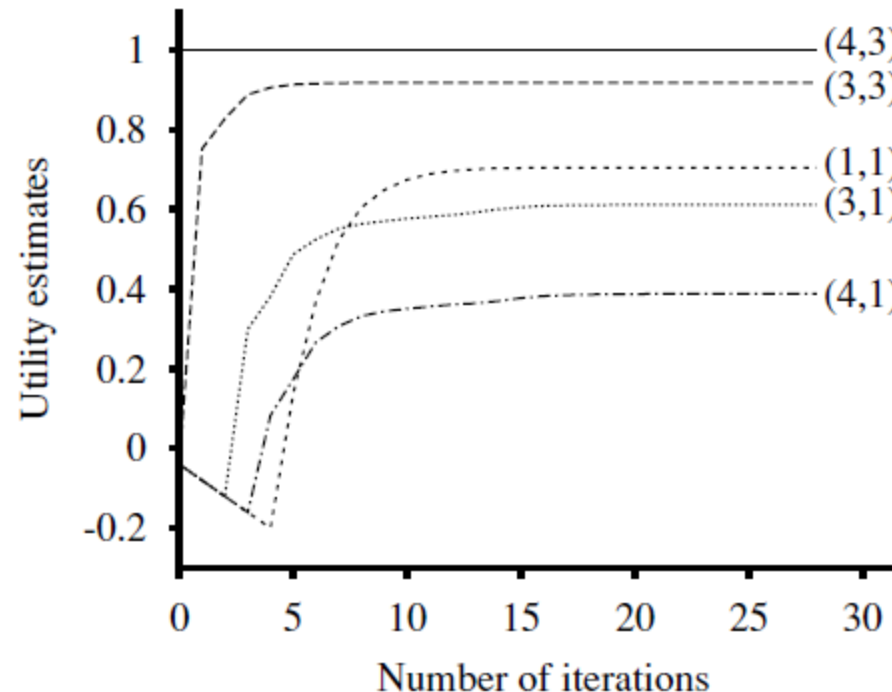
- Pseudocode of value iteration algorithm:

```
function VALUE-ITERATION( $mdp, \epsilon$ ) returns a utility function
  inputs:  $mdp$ , an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ ,
           rewards  $R(s)$ , discount  $\gamma$ 
            $\epsilon$ , the maximum error allowed in the utility of any state
  local variables:  $U, U'$ , vectors of utilities for states in  $S$ , initially zero
                     $\delta$ , the maximum change in the utility of any state in an iteration

  repeat
     $U \leftarrow U'; \delta \leftarrow 0$ 
    for each state  $s$  in  $S$  do
       $U'[s] \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$ 
      if  $|U'[s] - U[s]| > \delta$  then  $\delta \leftarrow |U'[s] - U[s]|$ 
  until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
  return  $U$ 
```

Markov Decision Process: Value Iteration Example

- Value iteration applied to grid world problem with initial utilities as 0s.
 - Note the number of iterations required for convergence of utilities for different states.



Markov Decision Process: Value Iteration Algorithm

- Does the value iteration converge to a solution?
- If apply Bellman update infinitely often, the equilibrium is guaranteed.
 - Final utility values must be the solutions to Bellman equations.
 - The solution is also unique.
 - *Proof of convergence:* We won't study in this course.
- Once utilities are obtained, we can also find the optimal policy! [*best action from each state satisfying Bellman equation*]

Markov Decision Process: Policy Iteration Algorithm

- Is it possible to get an optimal policy even when the utility function estimate is inaccurate?
 - *Answer: Yes.* If one action is clearly better than all others, then the exact magnitude of the utilities on the states involved need not be precise.
- An alternative approach can be designed using the above idea. This is known as **Policy Iteration.**

Markov Decision Process: Policy Iteration Algorithm

- **Steps of Policy Iteration Algorithm**

- Start with a random policy π_0

- Repeat until convergence:

- Policy Evaluation: Using the current policy π_i , calculate utilities U_i s as:

$$U_i(s) = U^{\pi_i}(s)$$

- Policy improvement: Using the utilities U_i 's, calculate a new updated policy π_{i+1} as:

$$\pi_{i+1}(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} U_i(s') P(s'|s, a)$$

- *Termination criteria*: The above algorithm terminates when policy improvement yields no change.

Markov Decision Process: Policy Iteration Algorithm

- How to evaluate the two steps?
 - *Policy improvement*: Pretty straightforward.
 - *Policy evaluation*: Not easy thought! [we discussed it previously]
 - *Solution?* Use an approach similar to the value iteration algorithm.
 - *One important difference*: Bellman equations for unities will be linear now! [why?]

Markov Decision Process: Policy Iteration Algorithm

- *Policy evaluation:*

- Utilities $U_i(s)$ of each state will satisfy the following simpler version of Bellman equation:

$$U_i(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$

- The equations are linear [*no max operation over actions as the policy is given and agent just follows the action given in the policy*]
 - We have n equations with n unknowns; all equations are linear.
 - Gaussian Elimination (or similar methods) will solve this.

Markov Decision Process: Policy Iteration Algorithm

- *Policy evaluation:*
 - Small state space: Gaussian elimination with $O(n^3)$ run-time.
 - For large space: Use modified policy iteration to find U 's
 - Same as policy iteration algorithm
 - Use small number of iterations as exact solution is not needed

Markov Decision Process: Policy Iteration Algorithm

▪ Pseudocode:

```
function POLICY-ITERATION( $mdp$ ) returns a policy
  inputs:  $mdp$ , an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ 
  local variables:  $U$ , a vector of utilities for states in  $S$ , initially zero
                    $\pi$ , a policy vector indexed by state, initially random

  repeat
     $U \leftarrow$  POLICY-EVALUATION( $\pi, U, mdp$ )
     $unchanged? \leftarrow$  true
    for each state  $s$  in  $S$  do
      if  $\max_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s'] > \sum_{s'} P(s' | s, \pi[s]) U[s']$  then do
         $\pi[s] \leftarrow \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) U[s']$ 
         $unchanged? \leftarrow$  false
  until  $unchanged?$ 
  return  $\pi$ 
```

Chapter 18 (AIAMA)

Learning From Examples

Chapter 18 (AIAMA)

Learning From Examples

Sukarna Barua

Associate Professor, CSE, BUET

Learning Agent

- An agent is **learning** if it improves its performance on future tasks after making observations about the world.
- Learning can be **online or offline**
 - Online: You learn as you observe each data; learn continuously over time.
 - Offline: You are given with a set of pre-observed data and you learn from the data; learn once.

Learning Agent

- **Input data:**

- Traffic info of several days in a month
- Data may have labels:
 - Rows are marked with “Good” and “Bad” traffic days

- **Learning algorithm:**

- Allow agent learns from the input data to predict new days as “Good” or “Bad”.

Learning types

- **Unsupervised:** Learn patterns in the input
 - Example: “Good traffic days” and “bad traffic days” learned by a taxi driver without any label to a data
 - Also known as **clustering**
 - Data do not have labels (Good/Bad).

Learning types

- **Supervised:**
 - Learn from input-output pairs.
 - Data have labels (i.e., class information)
 - Can predict “Good” or “Bad” from traffic info of a day

Learning types

- **Reinforcement:**
 - Learn from reinforcements, rewards and punishments
 - Example: lack of a tip at the end of a journey an indication that taxi driver did something wrong

Supervised Learning

- Given a **training set** of N example input–output pairs (x_1, y_1) , $(x_2, y_2), \dots (x_N, y_N)$ here each y_j was generated by an unknown function $y = f(x)$, discover a function h that approximates the true function f .
- **Hypothesis:** The function h is a hypothesis.
- **State space search:** Learning is a search through the space of possible hypotheses for one that will perform well, even on new examples beyond the training set.

Supervised Learning

- **Accuracy:** To measure the accuracy of a hypothesis we give it a test set of examples that are distinct from the training set.
- **Generalization:** We say a hypothesis generalizes well if it correctly predicts the value of y for novel examples.
- Sometimes the function f is stochastic—it is not strictly a function of x , and what we have to learn is a conditional probability distribution, $P(Y | x)$.

Supervised Learning

- **Classification:** When the output y is one of a finite set of values (such as sunny, cloudy or rainy), the learning problem is called classification, and is called Boolean or binary classification if there are only two values.
- **Regression:** When y is a number (such as tomorrow's temperature), the learning problem is called regression.
 - Technically, solving a regression problem is finding a conditional expectation or average value of y , because the probability that we have found exactly the right real-valued number for y is 0.

Supervised Learning

- What is the best hypothesis to choose from the hypotheses space? All are consistent (agrees with all data points)

(a) Linear

(b) 6-deg polynomial

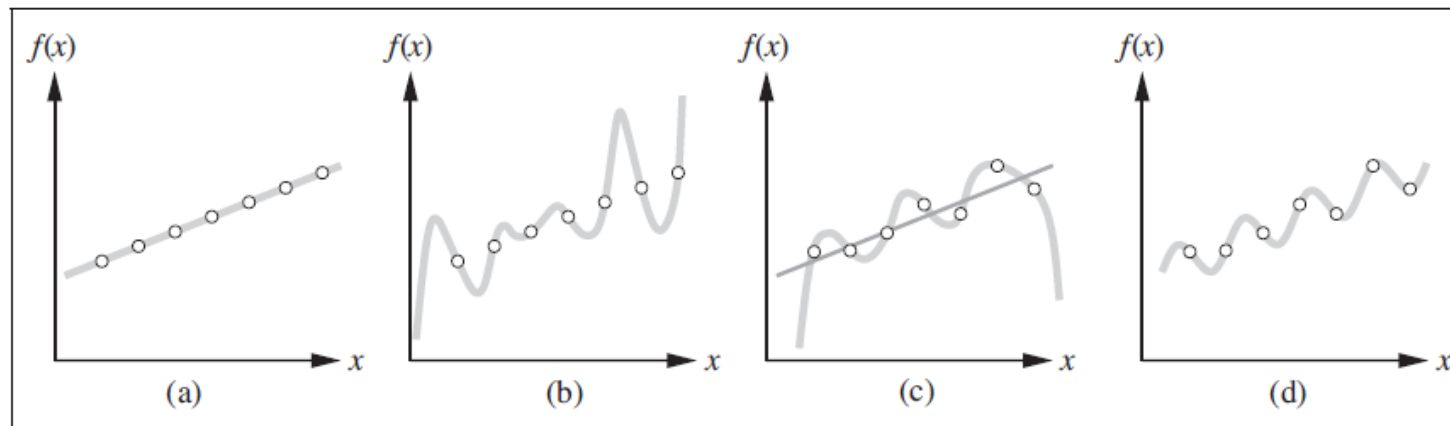
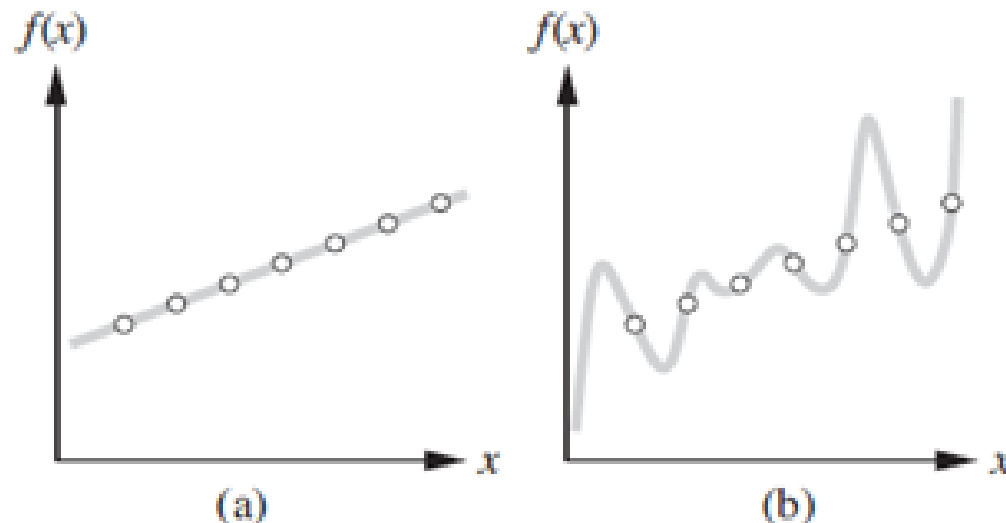


Figure 18.1 (a) Example $(x, f(x))$ pairs and a consistent, linear hypothesis. (b) A consistent, degree-7 polynomial hypothesis for the same data set. (c) A different data set, which admits an exact degree-6 polynomial fit or an approximate linear fit. (d) A simple, exact sinusoidal fit to the same data set.

Supervised Learning

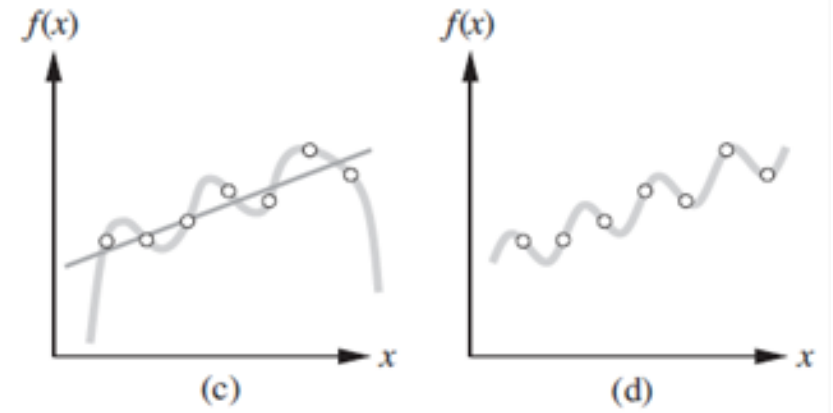
- **Occam's razor:** Select the simplest hypothesis consistent with the data.
 - After 14th-century English philosopher William of Ockham
 - Defining simplicity is not easy, but it seems clear that a degree-1 polynomial is simpler than a degree-7 polynomial, and thus (a) should be preferred to (b).
 - *Simplicity is evidence for truth* – Swinburne 1997



Supervised Learning

■ Consistency vs Generalization

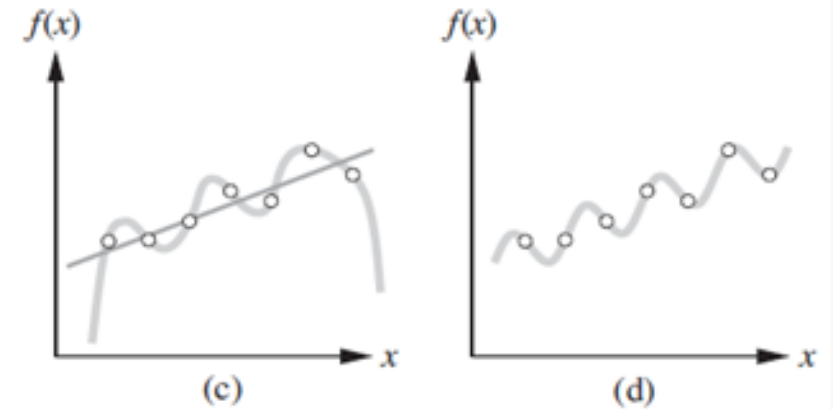
- Figure (c) shows a second data set.
- There is no consistent straight line for this data set;
- A degree-6 polynomial is an exact fit: may not find any pattern in the data and we do not expect it to generalize well.
- A straight line that is not consistent with any of the data points, but might generalize fairly well for unseen values of x .
- *In general, there is a tradeoff between complex hypotheses that fit the training data well and simpler hypotheses that may generalize better.*



Supervised Learning

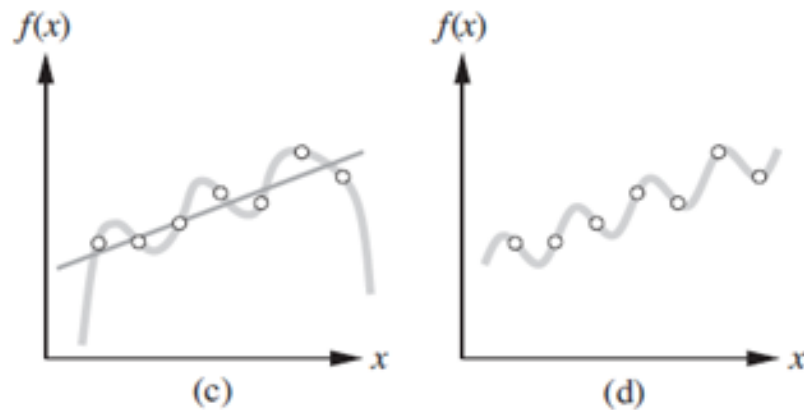
- **Choice of hypothesis space**

- In (d) we expand the hypothesis space H to allow polynomials over both x and $\sin(x)$, and find that the data in (c) can be fitted exactly by a simple function of the form $ax + b + c \sin(x)$.
- This shows the importance of the choice of hypothesis space.



Supervised Learning

- **Realizable:** We say that a learning problem is realizable if the hypothesis space contains the true function.
 - Unfortunately, we cannot always tell whether a given learning problem is realizable, because the true function is not known.



Supervised Learning

- **Example of Realizability:**

- **Linearly Separable Data:** If you are training a linear classifier (e.g., logistic regression or a perceptron) on a dataset that is perfectly linearly separable, then the problem is realizable because there exists a linear function that can classify the data with zero error.
- **Non-Linear Data:** If the data is not linearly separable, but you are restricted to using linear classifiers, the problem is non-realizable because no linear function can perfectly classify the data.

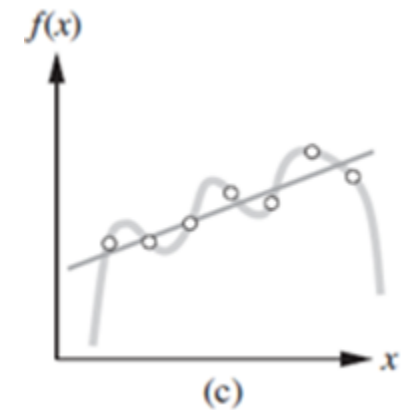
Supervised Learning

- **Selecting most probable hypothesis:** Supervised learning can be done by choosing the hypothesis h^* that is most probable given the data:

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h | \text{data}) .$$

By Bayes' rule this is equivalent to

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(\text{data} | h) P(h) .$$



- **Prior probability $P(h)$** is high for a degree-1 or 2 polynomial, lower for a degree-7 polynomial, and especially low for degree-7 polynomials with large spikes as in Figure (c).

Supervised Learning

- **Expressiveness vs computational complexity:** Why not let H be the class of all Java programs, or Turing machines?
 - After all, every computable function can be represented by some Turing machine, and that is the best we can do.
- **Two issues:**
 - Issue 1: Computation complexity of learning:** There is a tradeoff between the expressiveness of a hypothesis space and the complexity of finding a good hypothesis within that space. For example, fitting a straight line to data is an easy computation; fitting high-degree polynomials is somewhat harder; and fitting Turing machines is in general undecidable.

Supervised Learning

- **Expressiveness vs computational complexity:** Why not let H be the class of all Java programs, or Turing machines? After all, every computable function can be represented by some Turing machine, and that is the best we can do. Two issues:

Issue 2: Simpler hypothesis easier to use: A second reason to prefer simple hypothesis spaces is that presumably we will want to use h after we have learned it, and computing $h(x)$ when h is a linear function is guaranteed to be fast, while computing an arbitrary Turing machine program is not even guaranteed to terminate. For these reasons, most work on learning has focused on simple representations.

Chapter 18 (AIAMA)

Learning From Examples-02

Sukarna Barua

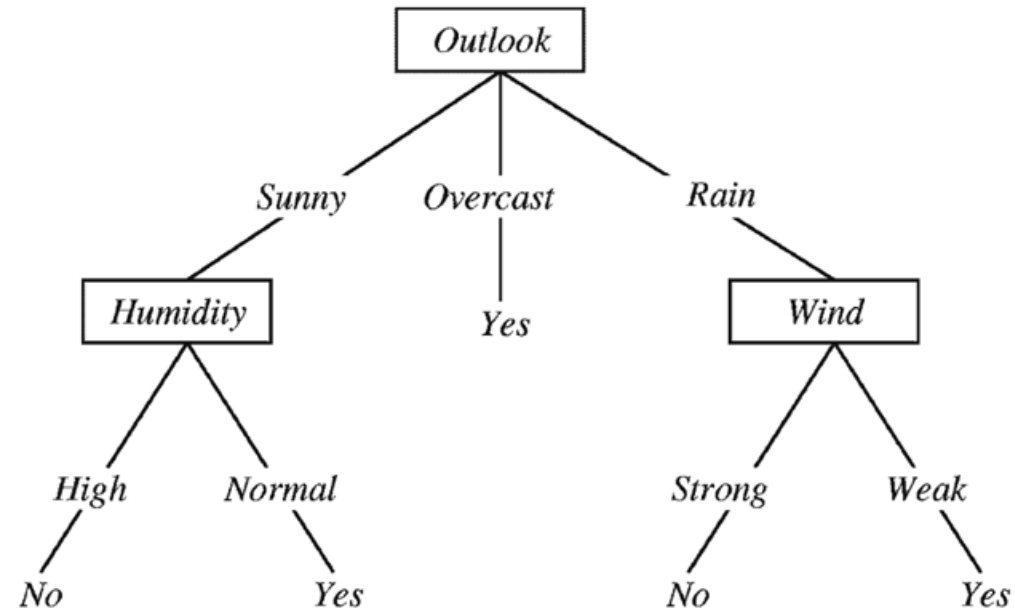
Associate Professor, CSE, BUET

Decision Tree

- **A decision tree:**
 - Represents a function that takes as input a vector of attribute values
 - Returns a “decision”—a single output value.
 - The input and output values can be discrete or continuous.
 - For now we will concentrate on problems where
 - Inputs have discrete values
 - Output has exactly two possible values; this is Boolean classification, where each example input will be classified as *true* (a **positive** example) or *false* (a **negative** example).

Decision Tree

- A decision tree for rain forecasting.
 - Input: {Outlook=Sunny, Humidity=Normal, Wind=Strong}
 - Output: Yes [Rainy]



Decision Tree

- **In a decision tree:**
 - Each **internal node** in the tree corresponds to a test of the value of one of the input attributes, A_i
 - The **branches** from the node are labeled with the possible values of the attribute, $A_i = v_{ik}$.
 - Each **leaf node** in the tree is marked with a outcome to be returned by the function.

Decision Tree: Example

- **Problem:** Build a decision tree to decide whether to wait at a restaurant.
- **Learning Goal:**
 - WillWait - to decide whether to wait for a table at a restaurant.
 - Goal is binary valued (i.e., binary classification task)
 - Values: {Yes, No}

Decision Tree: Example

- **Input Attributes**

1. *Alternate*: whether there is a suitable alternative restaurant nearby.
2. *Bar* : whether the restaurant has a comfortable bar area to wait in.
3. *Fri/Sat*: *true* on Fridays and Saturdays, *false* otherwise.
4. *Hungry*: whether we are hungry.
5. *Patrons*: how many people are in the restaurant (values are None, Some, and Full).
6. *Price*: the restaurant's price range (\$, \$\$, \$\$\$).

Decision Tree: Example

- **Input Attributes**

- 7. *Raining*: whether it is raining outside.

- 8. *Reservation*: whether we made a reservation.

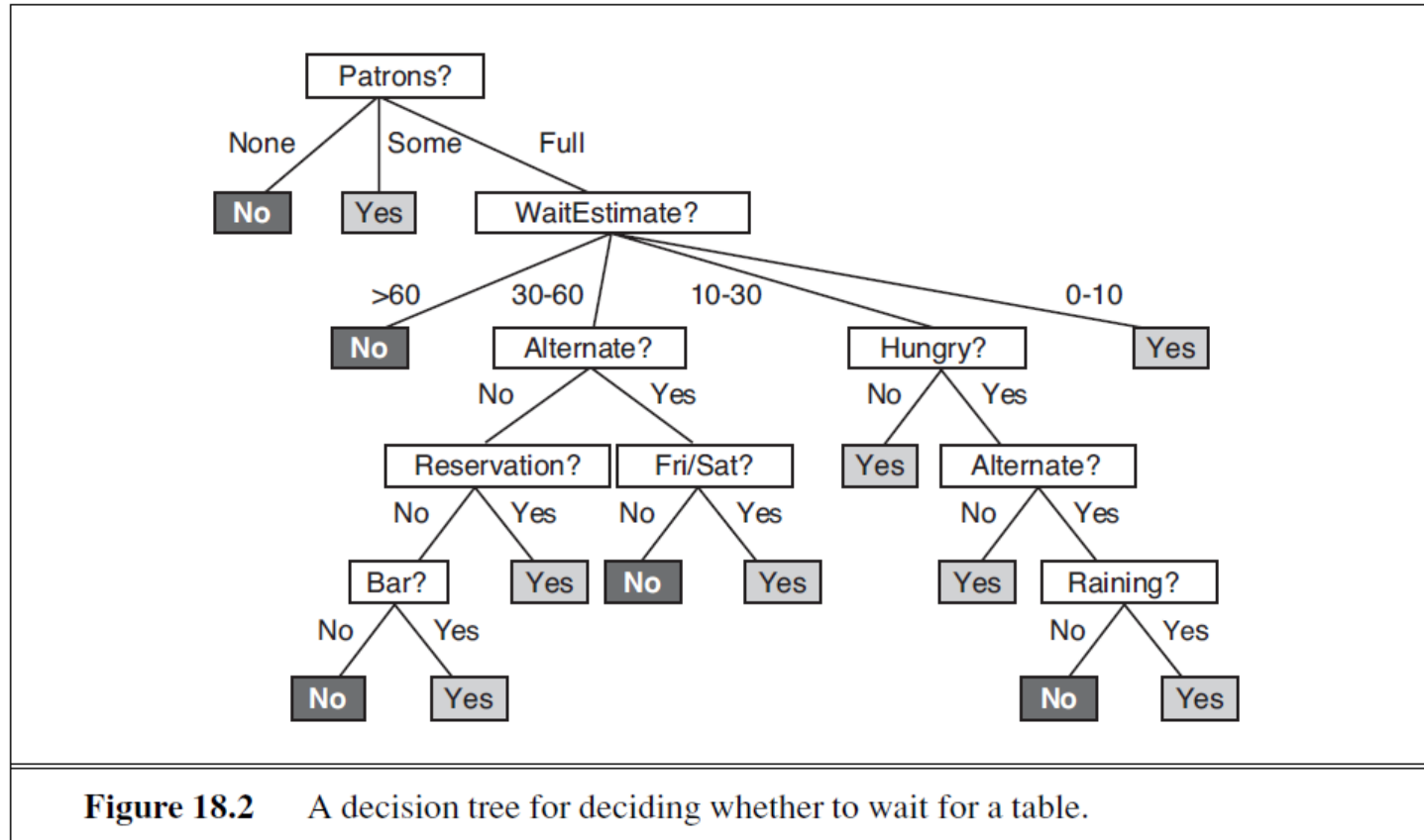
- 9. *Type*: the kind of restaurant (French, Italian, Thai, or burger).

- 10. *WaitEstimate*: the wait estimated by the host (0–10 minutes, 10–30, 30–60, or >60).

- Note that every variable has a small set of possible values; the value of *WaitEstimate*, for example, is not an integer, rather it is one of the four discrete values 0–10, 10–30, 30–60, or >60.

Decision Tree: Example

- An example decision tree for the restaurant problem



Expressiveness of Decision Trees

- In a Boolean decision tree, the goal attribute is true if and only if the input attributes satisfy one of the paths leading to a leaf with value true.
- $Goal \Leftrightarrow (Path1 \vee Path2 \vee \dots)$, where each $Path$ is a conjunction of attribute-value tests required to follow that path.
- Thus, the whole expression is equivalent to *disjunctive normal form*, which means that any function in propositional logic can be expressed as a decision tree.

Expressiveness of Decision Trees

- Any Boolean function can be represented by a decision tree.
 - Consider the following Boolean function: $f(A, B) = A + B$.
 - Draw a decision tree for the function:

Expressiveness of Decision Trees

- For a wide variety of problems, the decision tree format yields a nice, concise result.
- But some functions cannot be represented concisely:
 - For example, the majority function, which returns true if and only if more than half of the inputs are true, requires an exponentially large decision tree.
- Decision trees are good for some kinds of functions and bad for others.

Expressiveness of Decision Trees

- **How many different decision trees can be obtained for a Boolean function with n variables?**
 - A truth table over n attributes has 2^n rows, one for each combination of values of the attributes.
 - There are 2^{2^n} different functions.
 - Each function can be represented by a decision tree. Hence, number of decision trees at least 2^{2^n} [why more?]

Expressiveness of Decision Trees

- **How many different decision trees can be obtained for a Boolean function with n variables?**
 - For 10 Boolean attributes of our restaurant problem there are 2^{1024} or about 10^{308} different functions to choose from.
 - Number of possible decision trees $\geq 10^{308}$

Inducing Decision Trees

- **Training data:** A set of examples where each example is a pair $(\mathbf{x}, y)$ where $\mathbf{x}$ is a vector of values for the input attributes, y is a single Boolean output value.

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
$\mathbf{x}_1$	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	$y_1 = \text{Yes}$
$\mathbf{x}_2$	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	$y_2 = \text{No}$
$\mathbf{x}_3$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_3 = \text{Yes}$
$\mathbf{x}_4$	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	$y_4 = \text{Yes}$
$\mathbf{x}_5$	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
$\mathbf{x}_6$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	$y_6 = \text{Yes}$
$\mathbf{x}_7$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_7 = \text{No}$
$\mathbf{x}_8$	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	$y_8 = \text{Yes}$
$\mathbf{x}_9$	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
$\mathbf{x}_{10}$	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	$y_{10} = \text{No}$
$\mathbf{x}_{11}$	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	$y_{11} = \text{No}$
$\mathbf{x}_{12}$	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	$y_{12} = \text{Yes}$

Figure 18.3 Examples for the restaurant domain.

Inducing Decision Trees

- **Build a decision tree that is -**
 - Consistent with the examples [*Not always expected though, generalization may suffer*]
 - Is as small as possible. [*Occam's razor*]

Inducing Decision Trees

- **Question:** Can we always find a consistent decision tree given a set of training examples?.
- **Answer:** Yes.
 - However, if two training examples $(\mathbf{x}, y)$ and $(\mathbf{x}', y')$ have different outputs but same input attributes ($\mathbf{x} = \mathbf{x}'$), then a consistent decision tree is not possible.

Inducing Decision Trees

- Unfortunately, it is an intractable problem to find the smallest consistent tree; there is no way to efficiently search through *hypotheses space*.
 - An NP-hard problem!
- What can we do if cannot find the smallest decision tree?
- **Solution:** Use heuristics to find a closest one.
 - With some simple heuristics, we can find a good approximate solution: a small (*but not the smallest*) consistent tree.
 - This is a greedy approach. [Remember what is *greedy*]

Decision Tree Learning

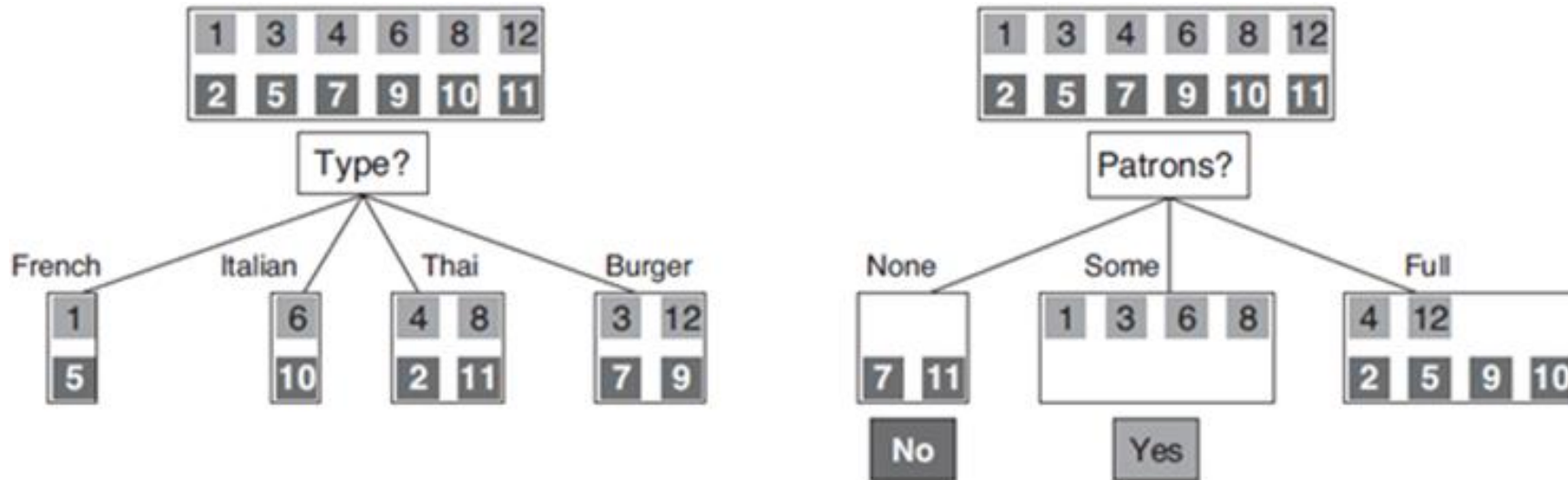
- **Greedy approach to build a decision tree:**
 - Start with empty decision tree.
 - Select an attribute to test at the next level [node]:
 - *Always select the most important attribute to test first. [greedy strategy]*
 - The test creates new branches and divides the problem into smaller subproblems.
 - Recurse on each child (created for each branch)

Decision Tree Learning

- **Greedy strategy:** *Always select the most important attribute to test first.*
 - *Most important attribute* implies the one that makes the most difference to the classification of an example. [*Get leaves as early as possible*]
 - Get correct classification with a small number of test
 - All paths in the tree will be short
 - Tree as a whole will be shallow.
 - *Above greedy strategy is a local optimal choice, may not necessarily leads to the globally smallest tree!*

Decision Tree Learning

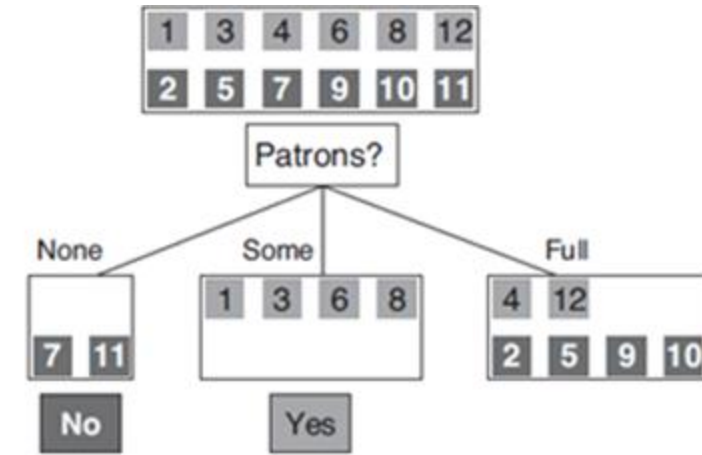
- Which attribute to test at root? Type vs Patron?
 - Type: all subsets (i.e., branches) needs further exploration.
 - Patron: two branches become leaves, only one needs further exploration.



Decision Tree Learning Algorithm

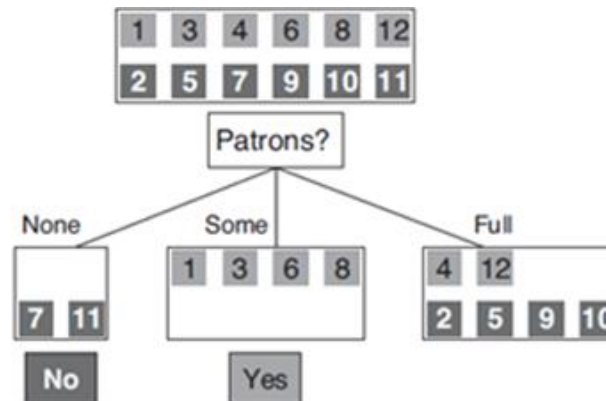
■ Decision tree construction:

- Step 1: Test an attribute at each node.
- Step 2: Partition the examples according to values and create child nodes with relevant examples.
- Step 3: Now consider child node for further tests of attributes except the one which have already been tested in the hierarchy (*recursive operation*)



Decision Tree Learning Algorithm

- **Four cases to consider at a child node:**
 - Case 1: All examples are positive (or all negative) → we are done; we can answer Yes or No. [e.g., *Patron=None*]
 - Case 2: Some positive and some negative examples → then choose the best attribute to split them recursively. [e.g., *Patron=Full*]



Decision Tree Learning Algorithm

- **Four cases to consider at a child node:**
 - Case 3: No examples left → No example has been observed for this combination of attribute values, and we return a default value (e.g., *plurality of parent node*)
 - Case 4: No attributes left, but both positive and negative examples remain → These examples have exactly the same description, but different classifications. This can happen because there is an error or **noise** in the data.
 - *In this case return the plurality classification of the remaining examples.*
[tree will not be consistent]

Decision Tree Learning Algorithm

- **Algorithm pseudocode**

```
function DECISION-TREE-LEARNING(examples, attributes, parent_examples) returns  
a tree  
  
  if examples is empty then return PLURALITY-VALUE(parent_examples)  
  else if all examples have the same classification then return the classification  
  else if attributes is empty then return PLURALITY-VALUE(examples)  
  else  
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$   
    tree  $\leftarrow$  a new decision tree with root test A  
    for each value  $v_k$  of A do  
       $\text{exs} \leftarrow \{e : e \in \text{examples} \text{ and } e.A = v_k\}$   
      subtree  $\leftarrow$  DECISION-TREE-LEARNING(exs, attributes – A, examples)  
      add a branch to tree with label (A =  $v_k$ ) and subtree subtree  
  return tree
```

- Function PLURALITY-VALUE selects the most common class/output among the examples

Choosing the Most Important Attribute

- **Which attribute is the most important now?**
 - **Perfect attribute:** One that splits into subsets where each subset contain either all positive or all negative examples. [*all branches become leaf nodes*]
 - **Useless attribute:** One that splits into subsets where each subset contain fairly equal mix of positive and negative examples. [*all branches need recursive exploration*]

Choosing the Most Important Attribute

- **Perfect vs. useless? How to measure?**
 - A formal measure of perfect vs useless: **Entropy**
 - The fundamental quantity in information theory (*Shannon and Weaver, 1949*).
 - Entropy is a measure of the uncertainty of a random variable.
 - Acquisition of information corresponds to a reduction in entropy.
 - A random variable with only one value—a coin that always comes up heads—has no uncertainty and thus its entropy is defined as zero; thus, we gain no information by observing its value.

Choosing the Most Important Attribute

- **Entropy:** Average number of bits per symbol to encode information.
- The roll of a fair *four*-sided die has 2 bits of entropy, because it takes two bits to describe one of four equally probable choices.
- An unfair coin that comes up heads 99% of the time.
 - This coin has less uncertainty than the fair coin—if we guess heads we'll be wrong only 1% of the time—it's entropy measure should be close to zero, but positive.

Entropy Measure

- In general, the entropy $H(V)$ of a random variable V with values v_k , each with probability $P(v_k)$, is defined as :

$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = - \sum_k P(v_k) \log_2 P(v_k)$$

Entropy Measure

- Verify that entropies measures are correct.

We can check that the entropy of a fair coin flip is indeed 1 bit:

$$H(\textit{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1 .$$

If the coin is loaded to give 99% heads, we get

$$H(\textit{Loaded}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08 \text{ bits.}$$

Entropy Measure

- Define $B(q)$ as the entropy of a Boolean random variable that is true with probability q :

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

- If a training set contains p positive examples and n negative examples, then the entropy of the goal attribute on the whole set is:

$$H(Goal) = B\left(\frac{p}{p + n}\right) .$$

Entropy Measure in Decision Tree

- **Decision tree contest:** Entropy represents an impurity measure of the set.
 - A set with 5 positive and 5 negative examples: Most impure, entropy should be highest.
 - A set with 10 positive and 0 negative examples: Purest, entropy should be the lowest.

Entropy Before Split

- A set with p positive and n negative examples.

- Entropy of the set is: $B\left(\frac{p}{p+n}\right)$

Entropy After Splitting

- An attribute A with d distinct values divides the training set E into subsets $E_1, E_2, \dots, E_d$.
- Each subset E_k has p_k positive examples and n_k negative examples, with entropy of $B(p_k/(p_k + n_k))$ bits of information. [E_k contains examples with $A = kth$ value]
- A randomly chosen example from the training set has the kth value for the attribute with probability $(p_k + n_k)/(p + n)$, so the expected entropy (*weighted average*) of the d subsets after splitting on attribute A is:

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

Information Gain

- **Information Gain:** The amount of information gain is the amount of entropy reduction after the split on attribute A.
- Assume E_{BS} = Entropy before split, E_{AS} = Entropy after split [*weighted average*]
 - Hence, information gain (= *reduction of entropy*):

$$Gain = E_{BS} - E_{AS} \text{ [reduction of entropy]}$$

- Hence, this is simply:

$$Gain(A) = B\left(\frac{p}{p+n}\right) - Remainder(A)$$

Information Gain

- **Information Gain:** $Gain(A) = B(\frac{p}{p+n}) - Remainder(A)$
- Compute the information gain for the attributes Patron and Type:

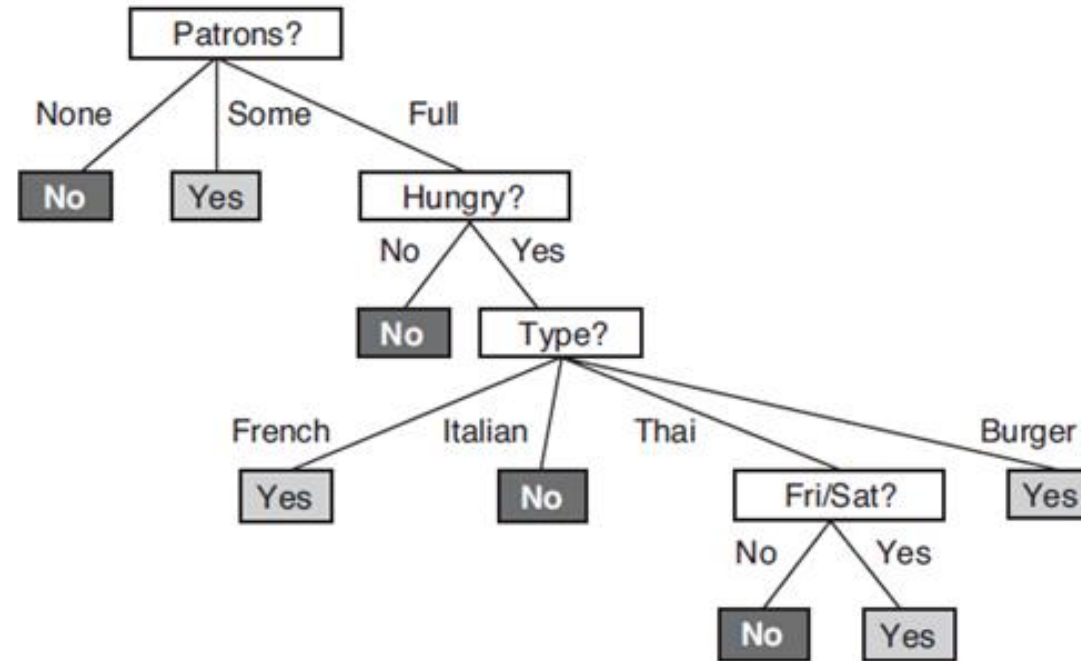
$$Gain(Patrons) = 1 - [\frac{2}{12}B(\frac{0}{2}) + \frac{4}{12}B(\frac{4}{4}) + \frac{6}{12}B(\frac{2}{6})] \approx 0.541 \text{ bits},$$

$$Gain(Type) = 1 - [\frac{2}{12}B(\frac{1}{2}) + \frac{2}{12}B(\frac{1}{2}) + \frac{4}{12}B(\frac{2}{4}) + \frac{4}{12}B(\frac{2}{4})] = 0 \text{ bits},$$

- Patron is a better attribute than Gain! Hence, choose Patron over Type.
- *Choose the attribute which gives the highest information gain!*

Final Decision Tree

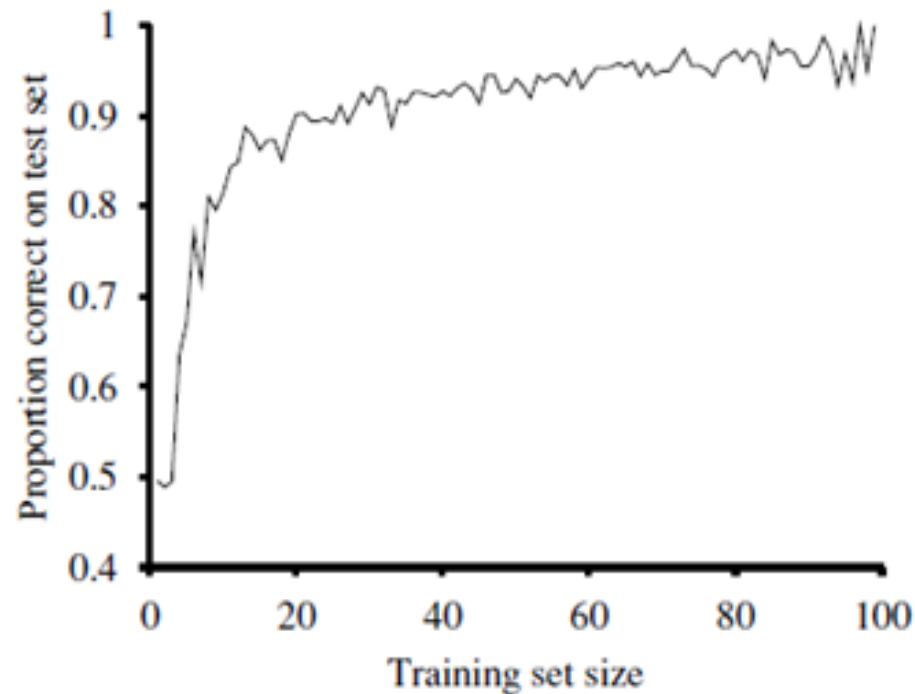
- Final decision tree constructed from given examples.



- Question:** What is the information gain for Type attribute at level 3?

Decision Tree Learning Curve

- Experiment with 100 examples: Construct decision trees
 - Split into train and test (e.g., 1 and 99, 2 and 98, etc.)
 - Random split 20 times and report average accuracy on test set
 - Note: As the training size grows, accuracy increases.



Overfitting

- **Overfitting:** Too much importance on every training example
 - Complex tree
 - Suffers generalization: very low training error, but very high test error.
 - Misses important concepts
- A consistent decision tree over training data may result in a complex tree
 - Noisy data may also induce complexity in the tree

Overfitting Solution

- **Pre-prune:** Prune the tree before it gets large
 - Early stopping: limit depth during tree construction
- **Post-pruning:** Prune the tree after construction
 - Remove irrelevant nodes
 - Replace internal nodes with most common class [*Only replace if test error do not increase*]

Decision Tree Issues

- **Missing values:** Some examples have missing value in some attributes
 - Solution: Replace missing value with mean of the attribute over the entire dataset
- **Continuous-valued attribute:** For example, height is continuous-valued.
 - Convert to categorical attribute:
 - Height > 40cm: *Tall* and Height <40cm: *Short*